



人工智能

陈科海

计算机科学与技术学院
哈尔滨工业大学（深圳）
chenkehai@hit.edu.cn



二、知识表示

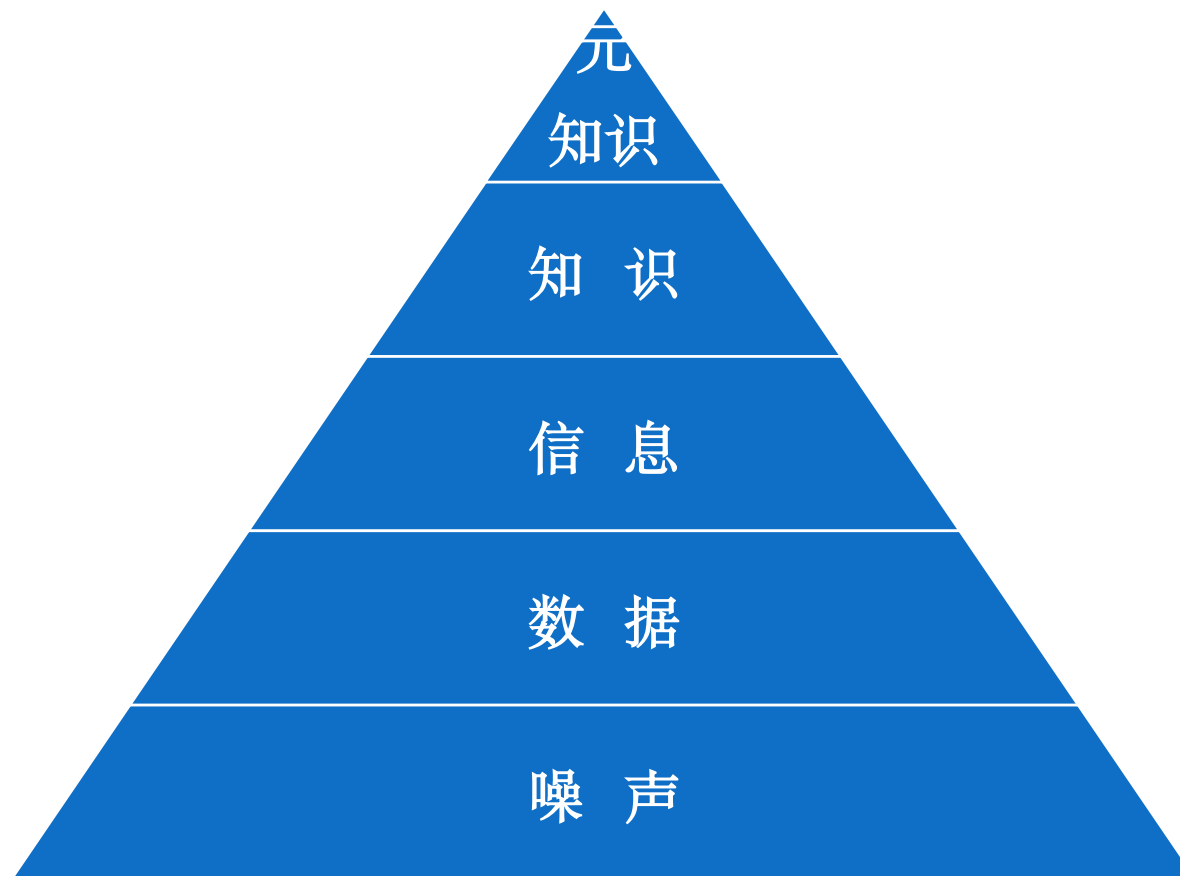
目 录

- 01 知识与知识表示基本概念
- 02 一阶谓词逻辑表示法
- 03 产生式规则表示法
- 04 框架表示法
- 05 语义网络和知识图谱表示法

2.1.1 知识与知识表示

□ 知识概念介绍

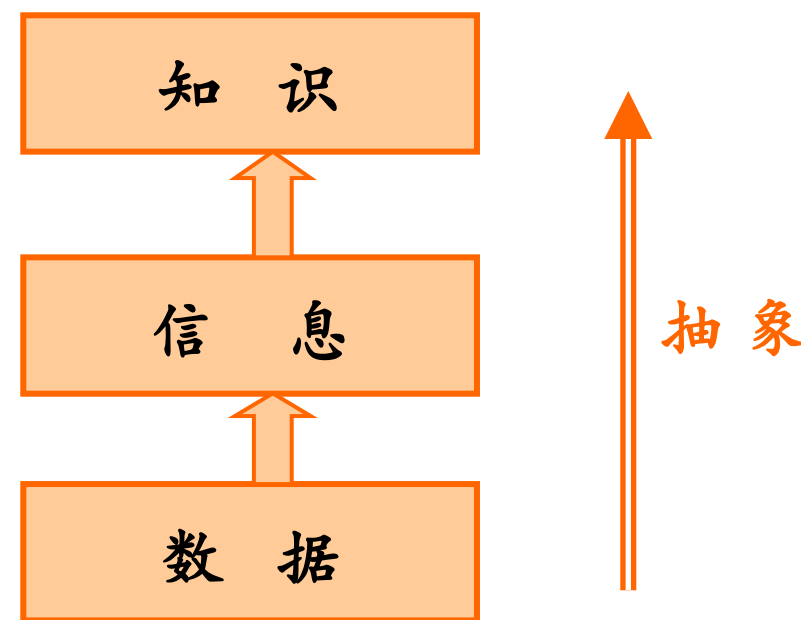
- 知识是人类智能的基础（符号主义学派）
- 智能活动过程主要是一个获取知识并运用知识的过程。
- 人工智能问题的求解也是以知识为基础的，知识的获取、知识的表示和运用知识进行推理是人工智能学科研究的3个主要问题。



2.1.1 知识与知识表示

□ 知识、信息和数据的关系

- **数据**：是记录信息的符号，是信息的载体和表示
- **信息**：是对数据的解释，是数据在具体的场合下具体的含义
- **知识**：一般把有关信息关联在一起所形成的信息结构称为知识
- **关系**：知识、信息、数据是**3**个层次的概念



2.1.1 知识与知识表示

□ 知识形成实例

137179766832525156430015

2.1.1 知识与知识表示

□ 知识形成实例

137179766832525156430015

- 
- ① 将每两位数字分为一组；
 - ② 忽略那些小于32的两位数；
 - ③ 把余下的每组两位数用**ASCII**字符代替。

13 71 79 76 68 32 52 51 56 43 00 15

2.1.1 知识与知识表示

□ 知识形成实例

137179766832525156430015

- 
- ① 将每两位数字分为一组；
 - ② 忽略那些小于32的两位数；
 - ③ 把余下的每组两位数用ASCII字符代替。

13 71 79 76 68 32 52 51 56 43 00 15

13 71 79 76 68 32 52 51 56 43 00 15



GOLD 438+

2.1.1 知识与知识表示

□ 知识形成实例

加工后的信息:

GOLD 438+

信息所表示的知识:

黄金价格为438，并且在升值(+)

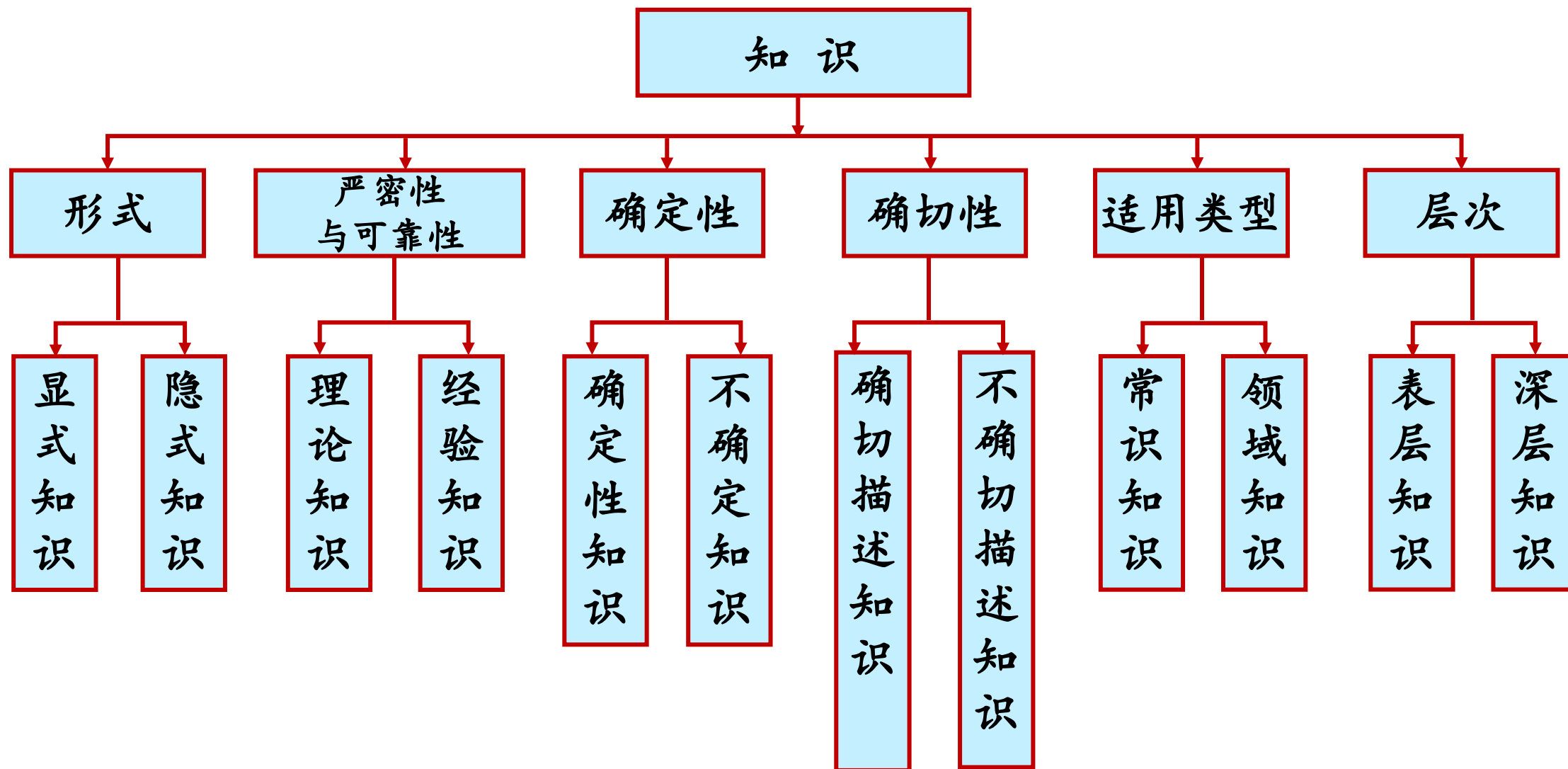
元知识(所谓元知识，就是指使用知识的知识):

如果：黄金价格低于500

且价格正在上涨 (+)

那么：购买黄金

2.1.2 知识的种类



2.1.3 知识的特性

➤ 相对正确性

例如: $1 + 1 = 2$

$$1 + 1 = 10$$

➤ 不确定性

知识并不总是只有“真”和“假”两种状态。引起知识不确定性的原因有:

- 1) 随机性
- 2) 模糊性
- 3) 不完全性
- 4) 经验性

2.1.3 知识的特性

➤ 相对正确性

例如: $1 + 1 = 2$

$1 + 1 = 10$

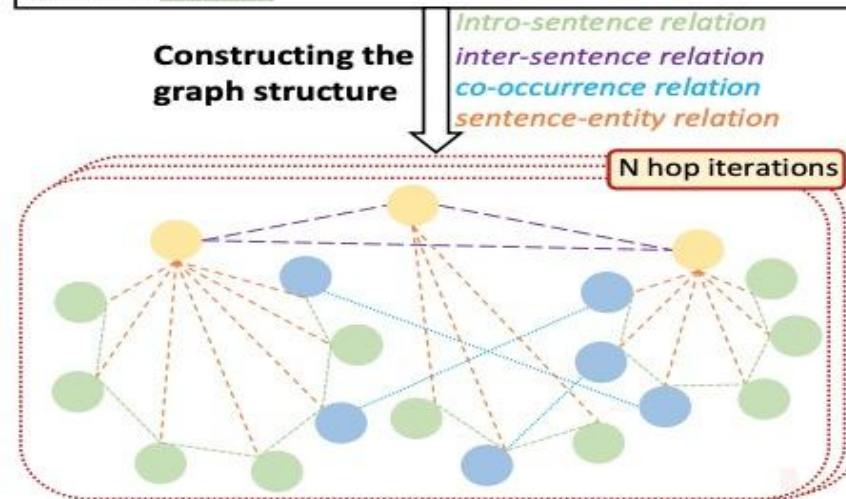
➤ 不确定性

知识并不总是只有“真”和“假”两种状态。引起知识不确定性的原因有:

- 1) 随机性
- 2) 模糊性
- 3) 不完全性
- 4) 经验性

➤ 可表示性

[1] Austin owns a scuba diving shop in Miami that provides diving training and sells a whole line of recreational scuba diving gear from fins to spearguns
[2] FBI agents of Florida recently paid a visit to him
[3] Like other scuba diving shop owners in Florida, Austin did not receive any specific instruction as to whom and what he should look for



➤ 可利用性

利用知识进行推理求解

2.1.4 知识的表示

□ 什么是知识表示

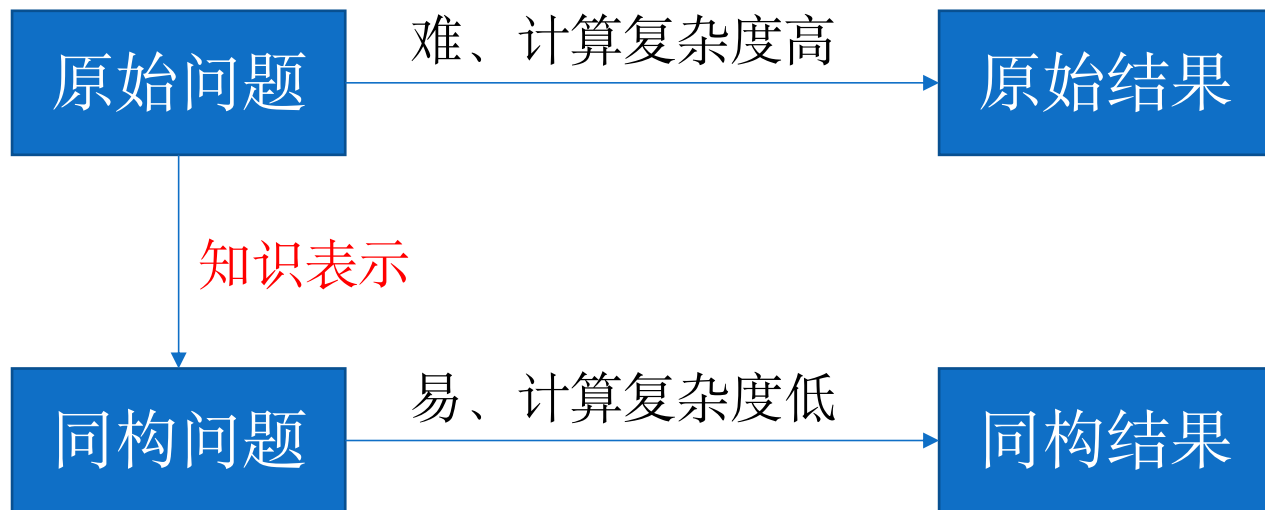
- 面向计算机的知识描述或表达的形式和方法。
- 知识表示的过程就是把知识编码成某种数据结构的过程。

2.1.4 知识的表示

□ 什么是知识表示

- 面向计算机的知识描述或表达的形式和方法。
- 知识表示的过程就是把知识编码成某种数据结构的过程。

□ 知识表示过程



2.1.4 知识的表示

□ 知识表示的要求

表示能力

可利用新

可组织性

可维护性

可实现性

自然性

可理解性

2.1.4 知识的表示

□ 知识表示的要求

表示能力

可利用新

可组织性

可维护性

可实现性

自然性

可理解性

□ 知识表示方法

谓词逻辑表示

产生式表示

语义网络表示

框架表示

目 录

01 知识与知识表示基本概念

02 一阶谓词逻辑表示法

03 产生式规则表示法

04 框架表示法

05 语义网络和知识图谱表示法

2.2.1 一阶谓词逻辑表示法

□ 谓词、函数、量词

➤ 谓词逻辑中的n元谓词:

$P(x_1, x_2, \dots, x_n)$

参量 (项/个体)

谓词符号 (大写字母)

2.2.1 一阶谓词逻辑表示法

□ 谓词、函数、量词

➤ 谓词逻辑中的n元谓词:

$P(x_1, x_2, \dots, x_n)$

参量 (项/个体)

谓词符号 (大写字母)

➤ 为了表达个体之间的对应关系,
引入n元个体函数, 简称函数:

$f(x_1, x_2, \dots, x_n)$

个体变元

函数符号 (小写字母)

2.2.1 一阶谓词逻辑表示法

□ 谓词、函数、量词

➤ 量 词：

“所有”、“一切”、“任一”、“全体”、“凡是”

全称量词，记为 $\forall x$

“存在”、“有些”、“至少有一个”、“有的”

存在量词，记为 $\exists x$

2.2.1 一阶谓词逻辑表示法

□ 【实例】关于量词的使用

(1) 所有的人都是要死的。

(2) 有的人活到100岁以上。

在个体域为人类集合时，可符号化为：

(1) $(\forall x)P(x)$ ，其中 $P(x)$ 表示 x 是要死的。

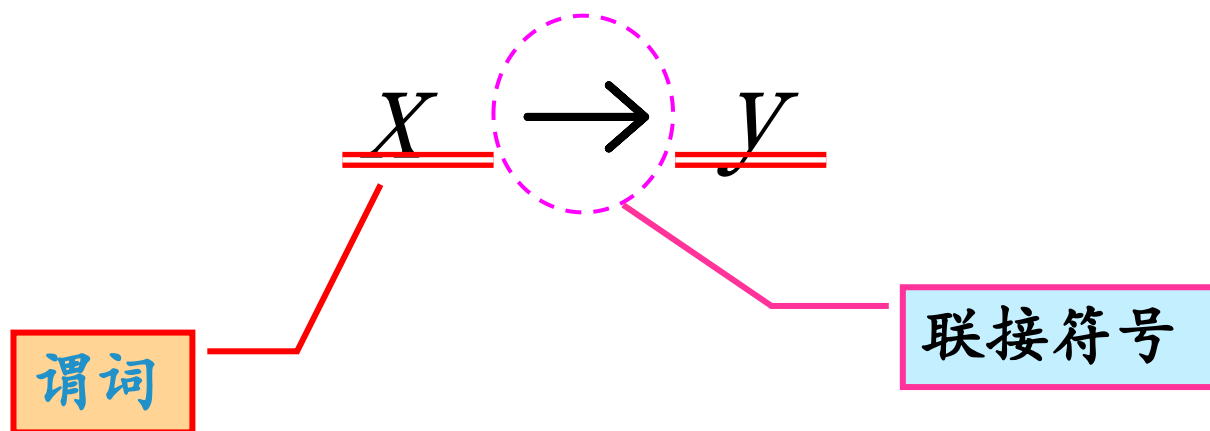
(2) $(\exists x)Q(x)$ ，其中 $Q(x)$ 表示 x 活到100岁以上。

2.2.2 谓词公式

用谓词联接符号将一些谓词联接起来所形成的公式。

例如：

$$\underline{\text{ISSTUDENT(张三)}} \wedge \underline{\text{ISSTUDENT(李四)}}$$



2.2.2 谓词公式

□ 常用的谓词联接符号

$\neg$: 否定 (非)

$\wedge$: 合取 (与)

$\vee$: 析取 (或)

$\rightarrow$: 蕴含 (IF... THEN)

$\leftrightarrow$: 等价 (当且仅当)

优先级



2.2.2 谓词公式

□ 【实例】联接符号的使用

$\neg$: “机器人不在2号房间”，可表示为：

$$\neg INROOM(Robot, R2)$$

$\wedge$: “我喜欢音乐和绘画”，可表示为：

$$LIKE(I, Music) \wedge LIKE(I, Painting)$$

$\vee$: “小张打篮球或踢足球”，可表示为：

$$PLAY(Zhang, Basketball) \vee PLAY(Zhang, football)$$

2.2.2 谓词公式

□ 辖域

紧接于量词之后被量词作用（即说明）的谓词公式称为该量词的辖域。

(1) 对于全称量词：

$$\forall x(P(x) \rightarrow G(x, y))$$

全称量词辖域

(2) 对于存在量词：

$$\exists x P(x) \wedge B(x)$$

存在量词辖域

2.2.3 指导变元、约束变元、自由变元

□ 指导变元

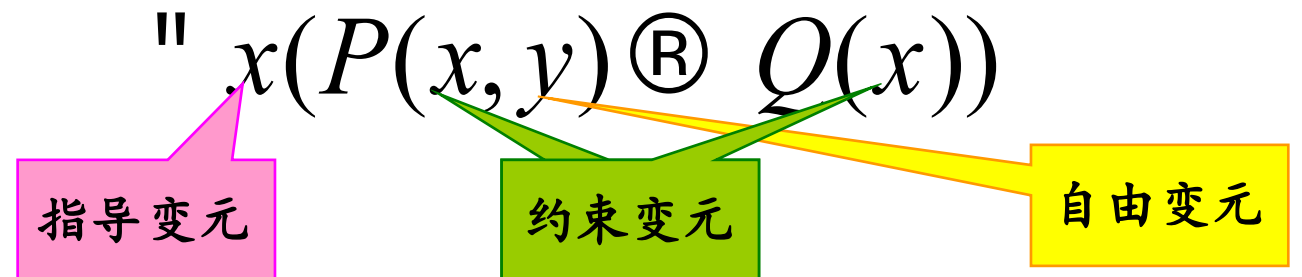
量词后面的变元称为量词的指导变元;

□ 约束变元

在一个量词的辖域中与该量词的指导变元相同的变元称为约束变元;

□ 自由变元

其它的变元称为自由变元



2.2.3 指导变元、约束变元、自由变元

□ 改名规则

一个变元在一个谓词公式中既可约束出现，又可自由出现，为了避免混淆，通常通过改名规则，使得一个谓词公式中一个变元仅以一种形式出现。

【换名规则】

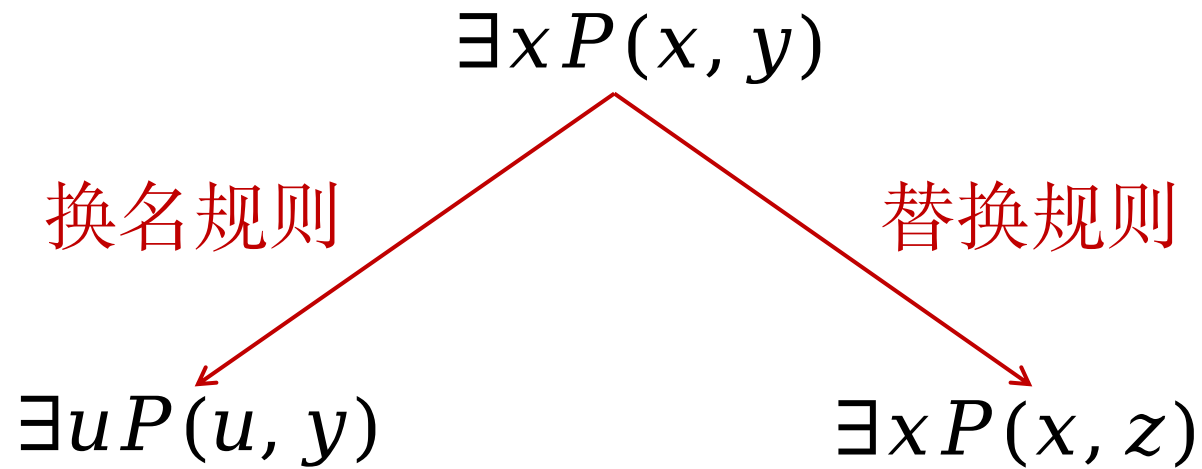
在谓词公式中，将某量词辖域中出现的某个**约束变元**以及对应的**指导变元**更改为本辖域中没有出现过的个体变元符号，公式其它部分不变，谓词公式的等价性不变。

【代替规则】

在谓词公式中，将某量词辖域中出现的某个**自由变元**的所有出现用本辖域中未曾出现过的某个个体变元符号代替，谓词公式的等价性不变。

2.2.3 指导变元、约束变元、自由变元

□ 改名规则的应用实例



2.2.3 指导变元、约束变元、自由变元

□ 改名规则的应用实例

$$\forall xP(x, y, z) \rightarrow \exists yQ(x, y, z)$$

利用换名规则：

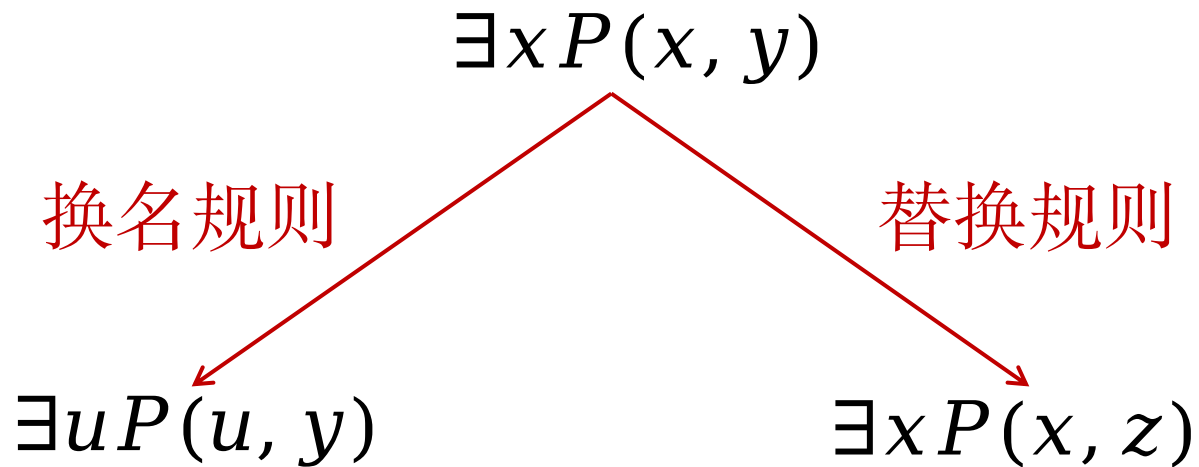
$$= \forall uP(u, y, z) \rightarrow \exists yQ(x, y, z)$$

$$= \forall uP(u, y, z) \rightarrow \exists vQ(x, v, z)$$

利用代替规则：

$$= \forall xP(x, u, z) \rightarrow \exists yQ(x, y, z)$$

$$= \forall xP(x, u, z) \rightarrow \exists yQ(v, y, z)$$



2.2.4 一阶谓词逻辑表示知识的步骤

- ① 定义谓词及个体，确定每个谓词及个体的确切含义；
- ② 根据所要表达的事物或概念，为每个谓词中的变元赋以特定的值；
- ③ 根据所要表达的知识的语义，用适当的联接符号将各个谓词联接起来，形成谓词公式。

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

例1：张三与李四打网球
(Zhang and Li play tennis)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

例1：张三与李四打网球
(Zhang and Li play tennis)

步骤1：

定义谓词： $\text{PLAY}(x, y, z)$ ：x和y进行运动z

定义个体： Zhang（张三）； Li（李四）； tennis（乒乓球）

步骤2：将个体代入谓词中

$\text{PLAY}(\text{Zhang}, \text{Li}, \text{tennis})$

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

- 例2:
- (1) 马科斯是男人。
 - (2) 马科斯是庞贝人。
 - (3) 所有庞贝人都是罗马人。
 - (4) 每个罗马人都有一个父亲。

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

- 例2: (1) 马科斯是男人。
(2) 马科斯是庞贝人。
(3) 所有庞贝人都是罗马人。
(4) 每个罗马人都有一个父亲。

- 解: (1) $MAN(Marcus)$;
(2) $POMPEIAN(Marcus)$;
(3) $\forall x ROMAN(x) \text{---} x$ 表示庞贝人
或者 $\forall x (POMPEIAN(x) \rightarrow ROMAN(x)) \text{---} x$ 表示人
(4) $\forall x (ROMAN(x) \rightarrow HASPARENT(x)) \text{---}$ 表示人
或者 $\forall x HASPARENT(x) \text{---} x$ 表示罗马人

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

例3:

- ① 有的人既喜欢梅花又喜欢菊花。
- ② 所有整数不是偶数就是奇数。
- ③ 并不是所有的学生选修了历史和生物。

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

定义谓词:

$LIKE(x, y)$: x 喜欢 y 。

定义个体:

x : 人

y : $meihua$ (梅花) , $juhua$ (菊花)

谓词公式表示为:

$$\exists x (LIKE(x, meihua) \wedge LIKE(x, juhua))$$

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

定义谓词:

$LIKE(x, y)$: x 喜欢 y 。

定义个体:

x : 人

y : *meihua* (梅花) , *juhua* (菊花)

谓词公式表示为:

$$\exists x (LIKE(x, meihua) \wedge LIKE(x, juhua))$$

定义谓词:

$I(x)$: x 是整数。

$E(x)$: x 是偶数

$O(x)$: x 是奇数

定义个体:

x : 整数

谓词公式表示为:

$$\forall x (I(x) \rightarrow E(x) \vee O(x))$$

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

定义谓词:

$TAKES(x, y)$: 学生 x 选修了课程 y

$STUDENT(x)$: x 是学生

定义个体:

H : 历史课程

B : 生物课程

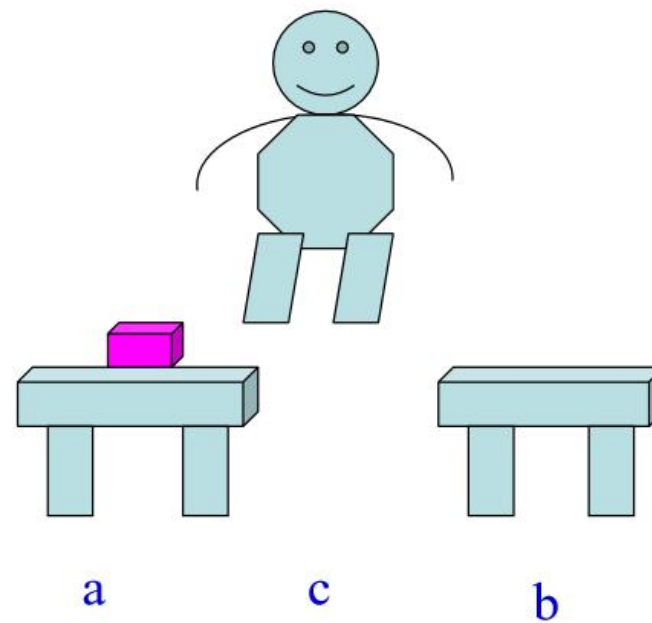
谓词公式表示为:

$$\neg(\forall x)(STUDENT(x) \rightarrow TAKES(x, H) \wedge TAKES(x, B))$$

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

例4：机器人移盒子



2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

例4：机器人移盒子

解：分别定义描述状态和动作的谓词

描述状态的谓词：

TABLE(x): x是桌子

EMPTY(y): y手中是空的

AT(y, z): y在z处

HOLDS(y, w): y拿着w

ON(w, x): w在x桌面上

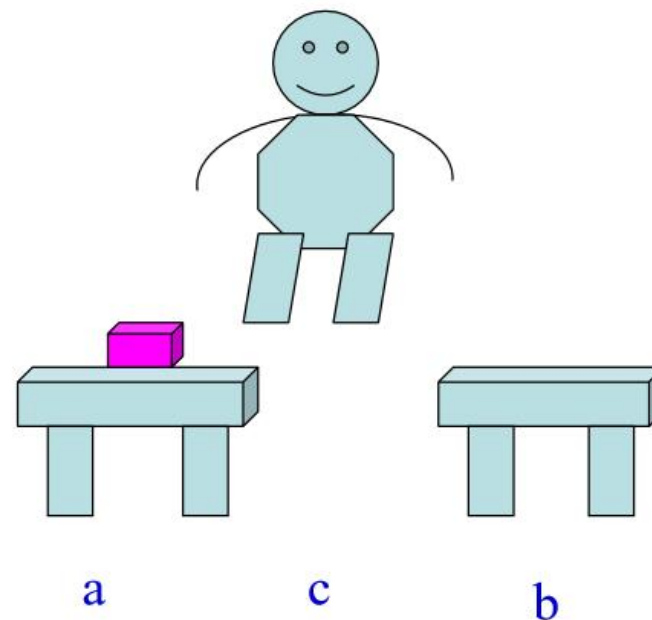
变元的个体域：

x的个体域是{a, b}

y的个体域是{robot}

z的个体域是{a, b, c}

w的个体域是{box}



2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

问题的初始状态:

AT(robot, c)
EMPTY(robot)
ON(box, a)
TABLE(a)
TABLE(b)



问题的目标状态:

AT(robot, c)
EMPTY(robot)
ON(box, b)
TABLE(a)
TABLE(b)

机器人行动的目标是把问题的初始状态转换为目标状态，而要实现问题状态的转换需要完成一系列的操作。

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

问题的初始状态:

AT(robot, c)
EMPTY(robot)
ON(box, a)
TABLE(a)
TABLE(b)



问题的目标状态:

AT(robot, c)
EMPTY(robot)
ON(box, b)
TABLE(a)
TABLE(b)

机器人行动的目标是把问题的初始状态转换为目标状态，而要实现问题状态的转换需要完成一系列的操作。

描述操作的谓词

条件部分: 用来说明执行该操作必须具备的先决条件，用谓词公式来表示。

动作部分: 给出了该操作对问题状态的改变情况，通过在执行该操作前的问题状态中删去和增加相应的谓词来实现。

这些操作包括:

Goto(x, y): 从x处走到y处。

Pickup(x): 在x处拿起盒子。

Setdown(y): 在x处放下盒子。

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

各操作的条件和动作：

Goto(x, y)

条件：AT(robot, x)

动作：删除表：AT(robot, x)

添加表：AT(robot, y)

Pickup(x)

条件：ON(box, x), TABLE(x), AT(robot, x), EMPTY(robot)

动作：删除表：EMPTY(robot), ON(box, x)

添加表：HOLDS(robot, box)

Setdown(x)

条件：AT(robot, x), TABLE(x), HOLDS(robot, box)

动作：删除表：HOLDS(robot, box)

添加表：EMPTY(robot), ON(box, x)

各操作的执行方法：

机器人每执行一操作前，都要检查该操作的先决条件是否可以满足。如果满足，就执行相应的操作；否则再检查下一个操作。

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

这个机器人行动规划问题的求解过程如下：

状态1(初始状态)

开始 AT(robot, c)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态2

 AT(robot, a)
Goto(c, a) EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态3

 AT(robot, a)
Pickup(a) HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

这个机器人行动规划问题的求解过程如下：

状态1(初始状态)

开始 AT(robot, c)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态2

 AT(robot, a)
Goto(c, a) EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态3

 AT(robot, a)
Pickup(a) HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

这个机器人行动规划问题的求解过程如下：

状态1(初始状态)

开始 AT(robot, c)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态2

Goto(c, a) AT(robot, a)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态3

Pickup(a) AT(robot, a)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

这个机器人行动规划问题的求解过程如下：

状态1(初始状态)

开始 AT(robot, c)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态2

Goto(c, a) AT(robot, a)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态3

Pickup(a) AT(robot, a)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

状态4

Goto(a, b) AT(robot, b)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

状态5

Setdown(b) AT(robot, b)
 EMPTY(robot)
=====> ON(box, b)
 TABLE(a)
 TABLE(b)

状态6(目标状态)

Goto(b, c) AT(robot, c)
 EMPTY(robot)
=====> ON(box, b)
 TABLE(a)
 TABLE(b)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

这个机器人行动规划问题的求解过程如下：

状态1(初始状态)

开始 AT(robot, c)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态2

Goto(c, a) AT(robot, a)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态3

Pickup(a) AT(robot, a)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

状态4

Goto(a, b) AT(robot, b)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

状态5

Setdown(b) AT(robot, b)
 EMPTY(robot)
=====> ON(box, b)
 TABLE(a)
 TABLE(b)

状态6(目标状态)

Goto(b, c) AT(robot, c)
 EMPTY(robot)
=====> ON(box, b)
 TABLE(a)
 TABLE(b)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

这个机器人行动规划问题的求解过程如下：

状态1(初始状态)

开始 AT(robot, c)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态2

Goto(c, a) AT(robot, a)
 EMPTY(robot)
=====> ON(box, a)
 TABLE(a)
 TABLE(b)

状态3

Pickup(a) AT(robot, a)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

状态4

Goto(a, b) AT(robot, b)
 HOLDS(robot, box)
=====> TABLE(a)
 TABLE(b)

状态5

Setdown(b) AT(robot, b)
 EMPTY(robot)
=====> ON(box, b)
 TABLE(a)
 TABLE(b)

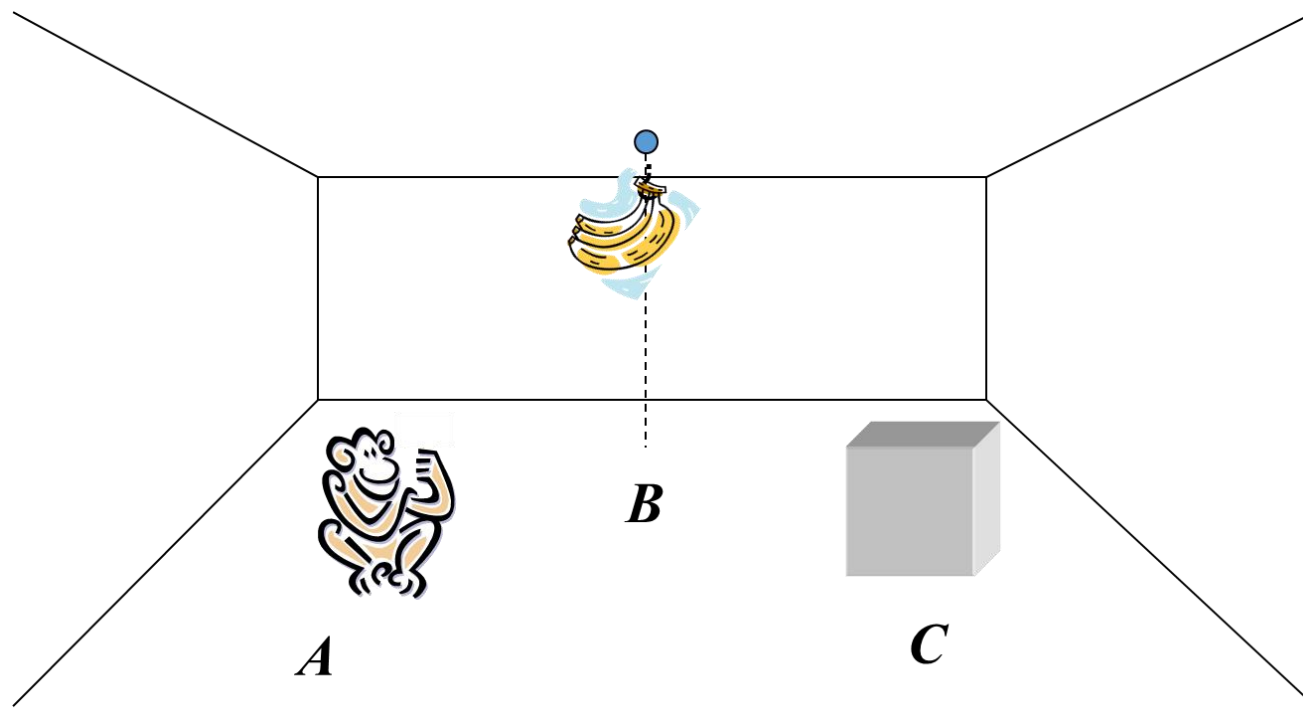
状态6(目标状态)

Goto(b, c) AT(robot, c)
 EMPTY(robot)
=====> ON(box, b)
 TABLE(a)
 TABLE(b)

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

例5: 假设房间里面有一只猴子（即机器人），位于**A**处。在**B**处上方的天花板上有一串香蕉，猴子想吃，但摘不到。房间**C**处还有一个箱子，如果猴子站在箱子上，就可以摸着天花板。



2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

解: 分别定义描述状态谓词

SITE(x, y): x在y 处;

HANG(w,y):w悬挂在y处

ON (z): z站在箱子上;

HOLDS(z): z手里拿着香蕉

变元的个体域:

x的个体域是{monkey, box}

y的个体域是{a, b, c}

z的个体域是{monkey}

w的个体域是{banana}

2.2.4 一阶谓词逻辑表示实例

□ 实例：一阶谓词逻辑表示

初始状态 S_0 :

$$\begin{aligned} S_0 = & SITE(Monkey, a) \\ & \wedge HANG(Banana, b) \\ & \wedge SITE(Box, c) \\ & \wedge \neg ON(Monkey) \\ & \wedge \neg HOLDS(Monkey) \end{aligned}$$

目标状态 S_g :

$$\begin{aligned} S_g = & SITE(Monkey, b) \\ & \wedge \neg HANG(Banana, b) \\ & \wedge SITE(Box, b) \\ & \wedge ON(Monkey) \\ & \wedge HOLDS(Monkey) \end{aligned}$$

定义四个操作:

Goto(u, v): 猴子从 u 走到 v 处。

Pushbox(v, w): 猴子推着箱子从 v 走到 w 处。

Climbbox: 猴子爬上箱子。

Grasp: 猴子摘到香蕉。

2.2.4 一阶谓词逻辑表示实例

各操作的条件和动作:

Goto(u,v)

条件: $\neg ON(Monkey), SITE(Monkey, u)$

动作: 删除表: $SITE(Monkey, u)$ 添加表: $SITE(Monkey, v)$

Pushbox(v,w)

条件: $\neg ON(Monkey), SITE(Monkey, v), SITE(Box, v)$

动作: 删除表: $SITE(Monkey, v), SITE(Box, v)$

添加表: $SITE(Monkey, w), SITE(Box, w)$

Climbbox

条件: $\neg ON(Monkey), SITE(Monkey, w), SITE(Box, w)$

动作: 删除表: $\neg ON(Monkey)$

添加表: $ON(Monkey)$

Grasp:

条件: $ON(Monkey), SITE(Box, b), HANG(Banana, b)$

动作: 删除表: $\neg HOLDS(Monkey), HANG(Banana, b)$

添加表: $HOLDS(Monkey), \neg HANG(Banana, b)$

2.2.4 一阶谓词逻辑表示实例

猴子摘香蕉求解过程:

$$\begin{aligned} S_0 = & \text{SITE}(\text{Monkey}, a) \\ & \wedge \text{HANG}(\text{Banana}, b) \\ & \wedge \text{SITE}(\text{Box}, c) \\ & \wedge \neg \text{ON}(\text{Monkey}) \\ & \wedge \neg \text{HOLDS}(\text{Monkey}) \end{aligned}$$

Goto(u, v)



$$\begin{aligned} S_1 = & \text{SITE}(\text{Monkey}, c) \\ & \wedge \text{HANG}(\text{Banana}, b) \\ & \wedge \text{SITE}(\text{Box}, c) \\ & \wedge \neg \text{ON}(\text{Monkey}) \\ & \wedge \neg \text{HOLDS}(\text{Monkey}) \end{aligned}$$

Pushbox(v, w)



$$\begin{aligned} S_2 = & \text{SITE}(\text{Monkey}, b) \\ & \wedge \text{HANG}(\text{Banana}, b) \\ & \wedge \text{SITE}(\text{Box}, b) \\ & \wedge \neg \text{ON}(\text{Monkey}) \\ & \wedge \neg \text{HOLDS}(\text{Monkey}) \end{aligned}$$

2.2.4 一阶谓词逻辑表示实例



$$S_g = SITE(Monkey, b)$$

$$\wedge \neg HANG(Banana, b)$$

$$\wedge SITE(Box, b)$$

$$\wedge ON(Monkey)$$

$$\wedge HOLDS(Monkey)$$

Grasp

$$S_2 = SITE(Monkey, b)$$

$$\wedge HANG(Banana, b)$$

$$\wedge SITE(Box, b)$$

$$\wedge \neg ON(Monkey)$$

$$\wedge \neg HOLDS(Monkey)$$

Climbbox

$$S_3 = SITE(Monkey, b)$$

$$\wedge HANG(Banana, b)$$

$$\wedge SITE(Box, b)$$

$$\wedge ON(Monkey)$$

$$\wedge \neg HOLDS(Monkey)$$

2.2.5 谓词逻辑表示法的特点

□ 优点

- (1) **严密性**：可以保证其演绎推理结果的正确性，可以较精确的表达知识。
- (2) **自然性**：谓词逻辑是一种接近于自然语言的形式语言。
- (3) **通用性**：拥有通用的逻辑演算方法和推理的规则。
- (4) **易于实现**：用它表示的知识易于模块化，便于知识的增删及修改，便于在计算机上实现。

2.2.5 谓词逻辑表示法的特点

□ 局限性

- (1) **知识表示能力差**：不便于表达和加入非确定性、启发性知识等。
- (2) **组合爆炸**：在其推理过程中，随着事实数目的增大及盲目的使用推理规则，有可能形成组合爆炸。
- (3) **效率低**：由于推理是根据形式逻辑进行的，把推理演算与知识含义截然分开，抛弃了表达内容中所含有的语义信息，往往使推理过程太冗长，降低了系统的效率。

目 录

- 01 知识与知识表示
- 02 一阶谓词逻辑表示法
- 03 产生式表示法
- 04 语义网络表示法
- 05 框架表示法

2.3 产生式表示法

□ 产生式简介

- 1943年Post 首先在一种计算形式体系中提出
- 形式上很简单，但在一定意义上模仿了人类思考的过程
- 60年代开始，成为专家系统基本的知识表示方法
- 适合表示事实性知识和规则性知识
- 容易描述事实、规则以及它们的不确定性度量

2.3.1 知识的产生式表示方法

□ 事实的表示

➤ 事实的概念

事实是断言一个语言变量的值或断言多个语言变量之间关系的陈述句。

➤ 语言变量的值: 例如, “雪是白的”

➤ 语言变量之间的关系: 例如, “王峰热爱祖国”

2.3.1 知识的产生式表示方法

□ 事实的表示

➤ 事实的表示

❖ 确定性知识

(对象, 属性, 值), 例如, (snow, color, white) 或(雪, 颜色, 白)。其中, 对象就是语言变量。

(关系, 对象1, 对象2), 例如, (love, Wang Feng, country) 或(热爱, 王峰, 祖国)

❖ 非确定性知识

(对象, 属性, 值, 可信度因子)

其中, "可信度因子"是指该事实为真的相信程度。可用 $[0, 1]$ 之间的一个实数来表示。

2.3.1 知识的产生式表示方法

□ 规则的表示

➤ **概念：** 规则的产生式表示形式常称为产生式规则，简称产生式或规则

➤ **产生式的基本形式**

$P \rightarrow Q$

或者 **IF P THEN Q**

前 提
(可用的条件)

结 论
(应该执行的操作)

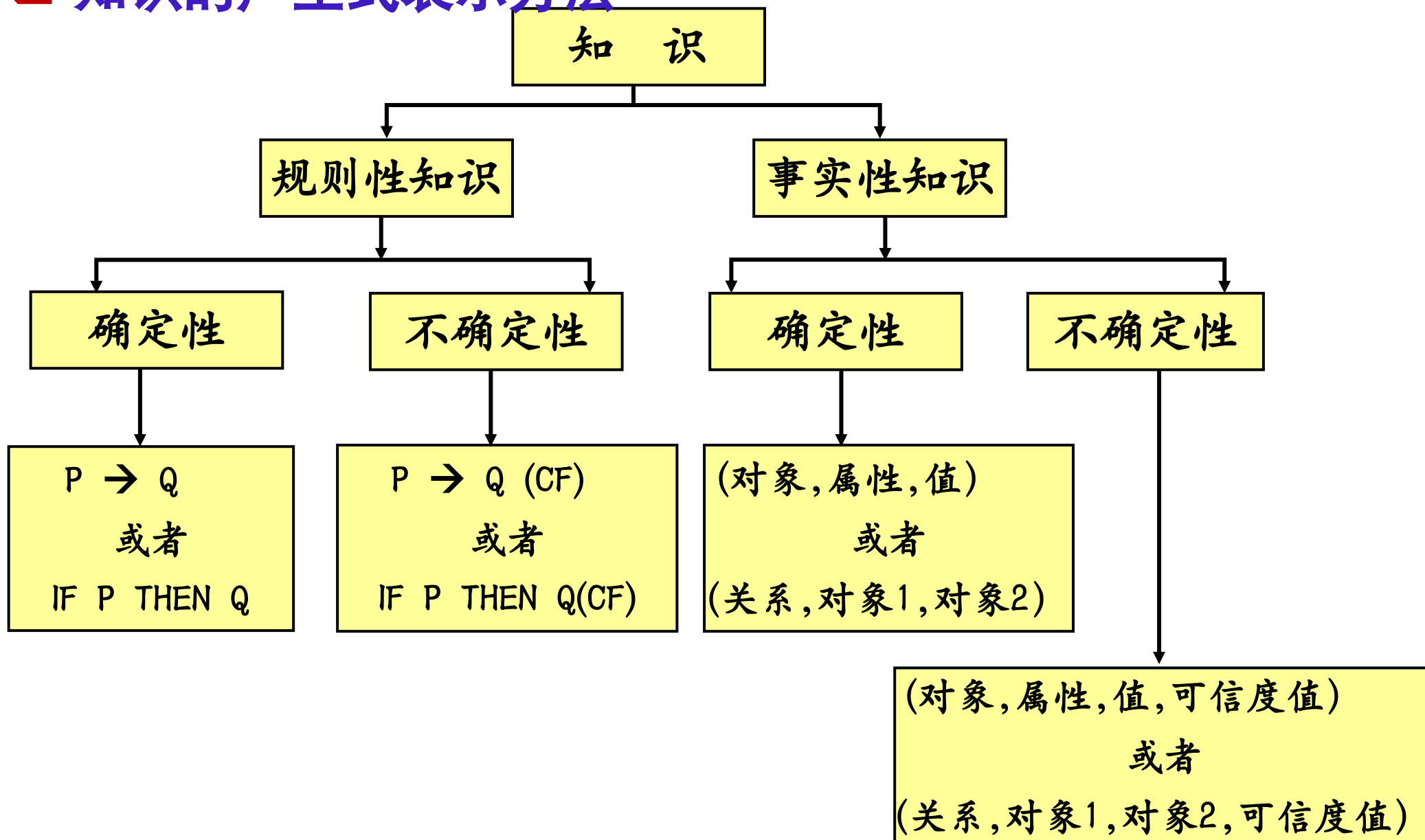
其中， **P**是产生式的前提，也称为前件，它给出了该产生式可否使用的先决条件。**Q**是一组结论或操作，也称为后件，它指出当**P**满足时，应该推出的结论或应该执行的动作。

➤ **产生式的简例：** "如果王宏是计算机系学生， 则王宏会编程序"

可用产生式表示为:**IF 该学生是计算机专业 THEN 该学生会编程序**

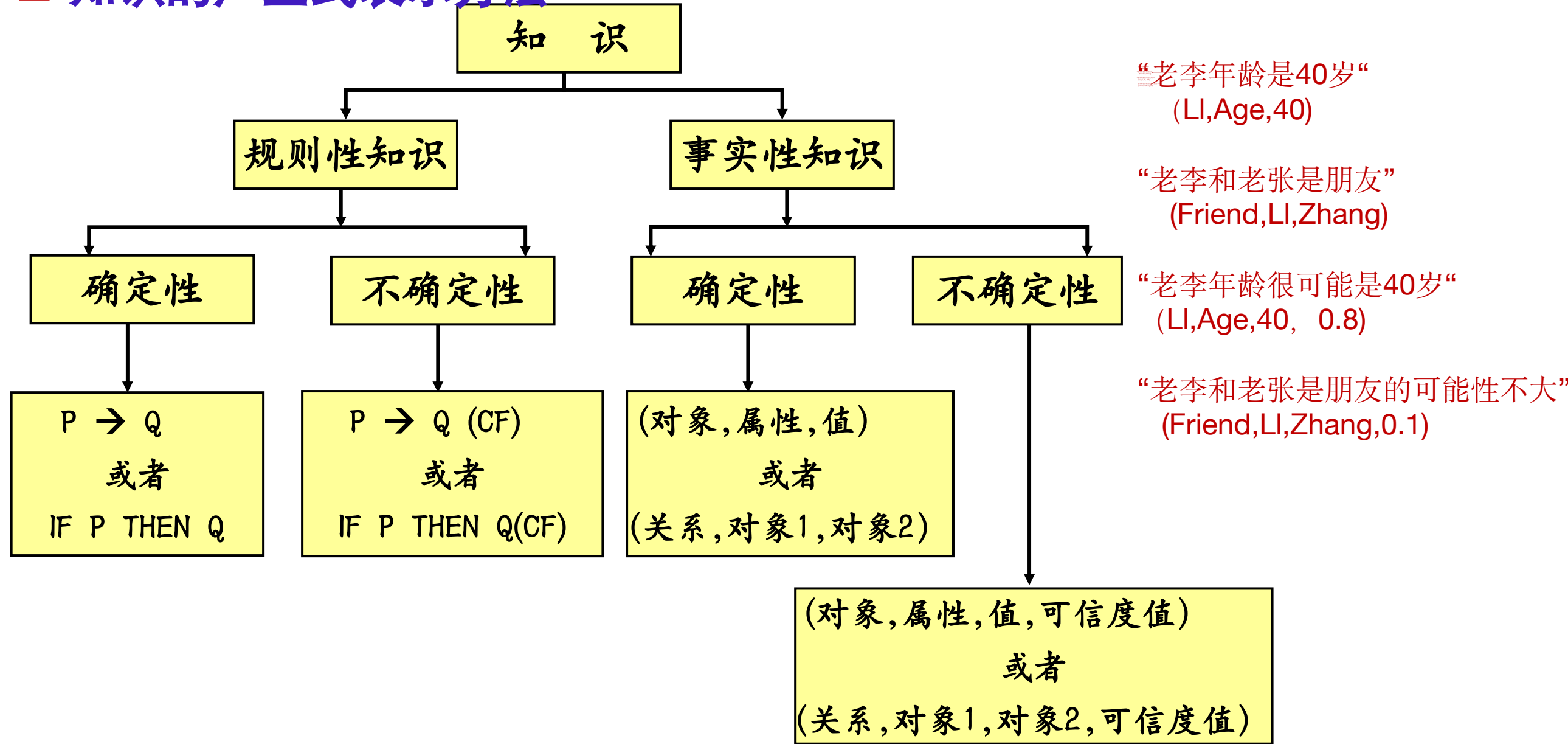
2.3.1 知识的产生式表示方法

□ 知识的产生式表示方法

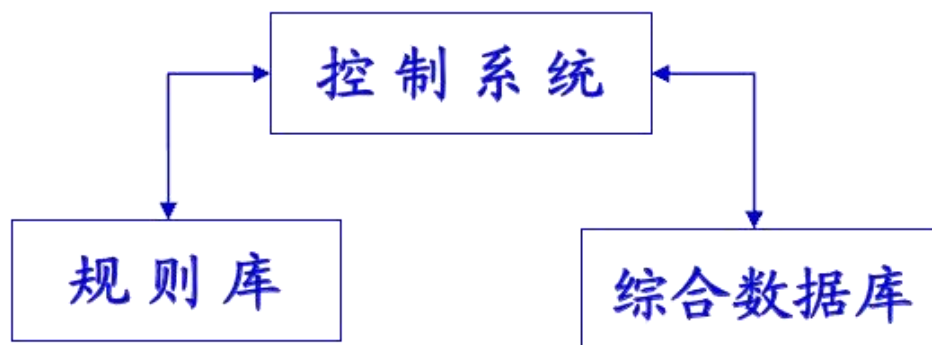


2.3.1 知识的产生式表示方法

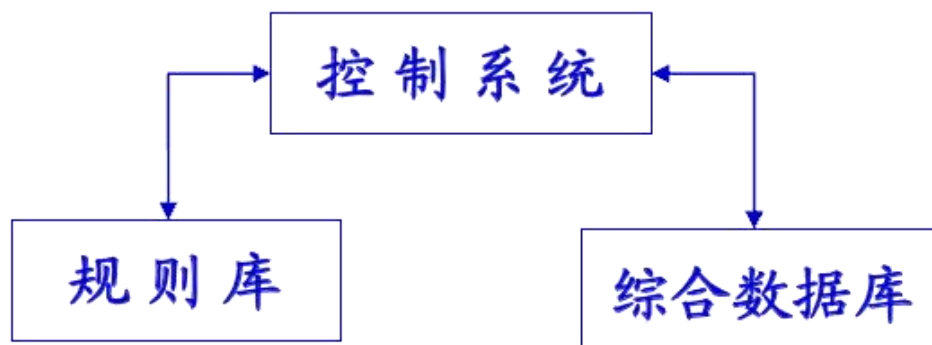
知识的产生式表示方法



2.3.2 产生式系统的基本结构



2.3.2 产生式系统的基本结构



➤ 综合数据库DB(Data Base)

❖ 存放推理过程的各种当前信息

比如：问题的初始状态；输入的事实；
中间结论及最终结论

❖ 作为推理过程选择可用规则的依据

推理过程中某条规则是否可用，是通过该规则的前提与DB中的已知事实的匹配来确定的。可匹配的规则称为可用规则。利用可用规则进行推理，将会得到一个结论。该结论若不是目标，将作为新的事实放入DB，成为以后推理的已知事实。

➤ 规则库RB(Rule/Knowledge Base)

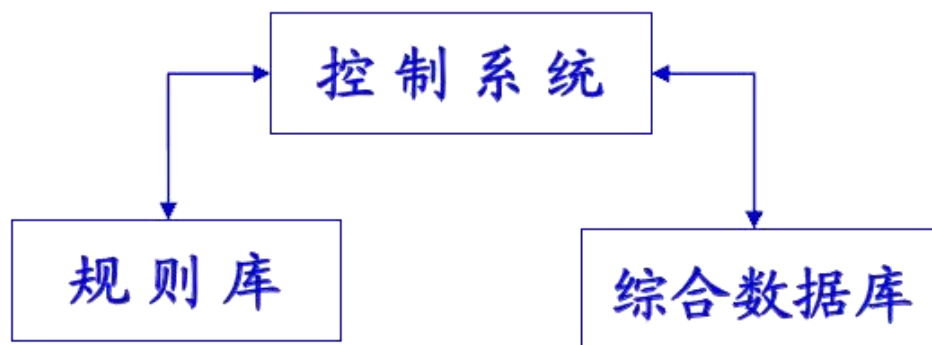
❖ 作用

用于存放推理所需要的所有规则，是整个产生式系统的知识集，是产生式系统能够进行推理的根本。

❖ 要求

知识的完整性、一致性、准确性、灵活性和可组织性

2.3.2 产生式系统的基本结构



➤ 控制系统(Control system)

❖ 控制系统的主要作用

亦称推理机，用于控制整个产生式系统的运行，决定问题求解过程的推理线路。

❖ 控制系统的主要任务

选择匹配：按一定策略从规则库中选择规则与综合数据库中的已知事实进行匹配。匹配是指把所选规则的前提与综合数据库中的已知事实进行比较，若事实库中存的事实与所选规则前提一致，则称匹配成功，该规则为可用；否则，称匹配失败，该规则不可用。

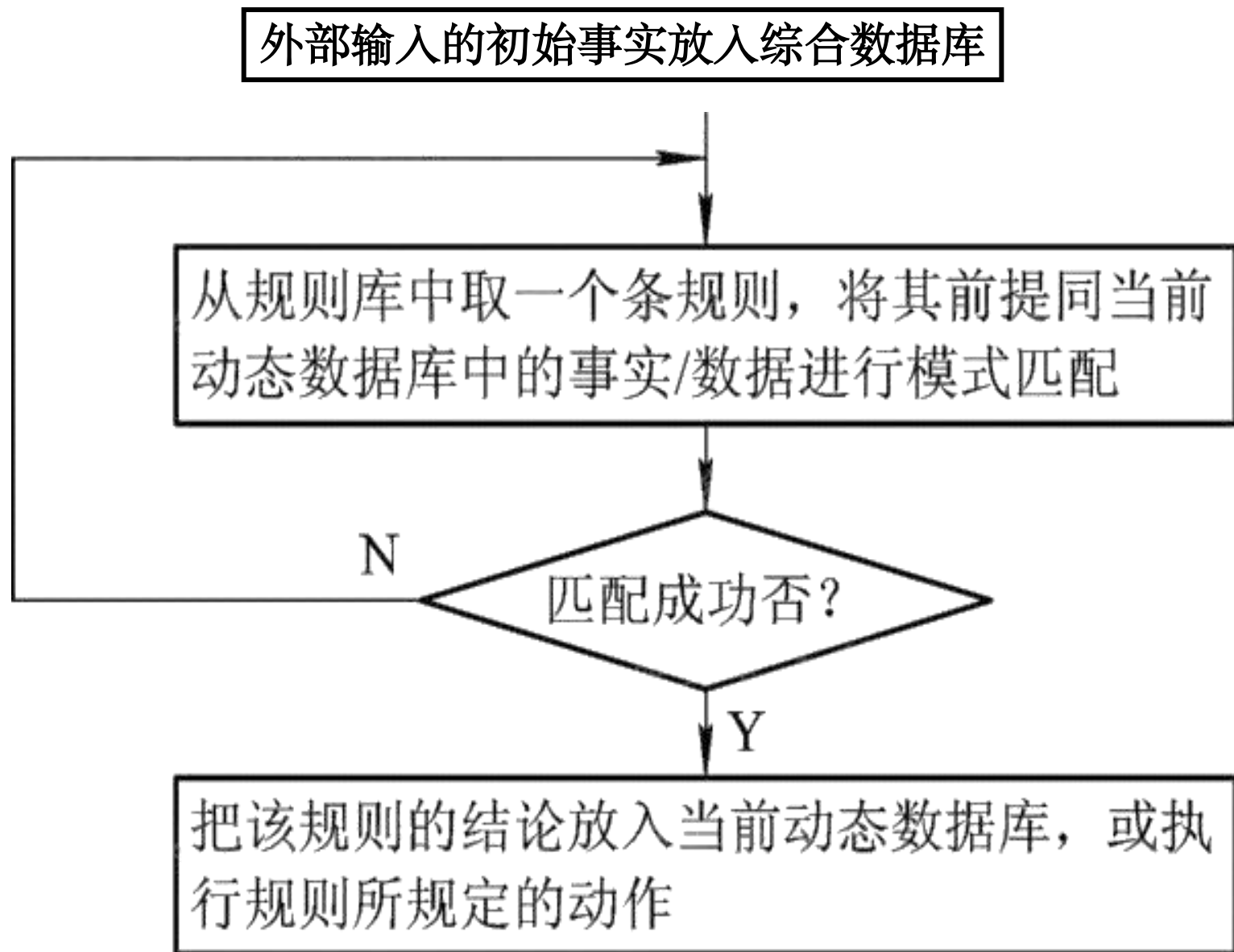
冲突消解：对匹配成功的规则，按照某种策略从中选出一条规则执行。

执行操作：对所执行的规则，若其后件为一个或多个结论，则把这些结论加入综合数据库；若其后件为一个或多个操作时，执行这些操作。

终止推理：检查综合数据库中是否包含有目标，若有，则停止推理。

路径解释：在问题求解过程中，记住应用过的规则序列，以便最终能够给出问题的解的路径。

2.3.3 产生式系统的运行过程



2.3.4 产生式系统应用举例

例1:设有以下两条规则

r_1 : IF 动物有羽毛 THEN 动物是鸟

r_2 : IF 动物是鸟 AND 动物善飞 THEN 动物是信天翁

其中, r_1 和 r_2 是上述两条规则在动物识别系统中的规则编号。

假设已知有以下事实:

动物有羽毛, 动物善飞

求满足以上事实的动物是何种动物。

2.3.4 产生式系统应用举例

例1:设有以下两条规则

r_1 : IF 动物有羽毛 THEN 动物是鸟

r_2 : IF 动物是鸟 AND 动物善飞 THEN 动物是信天翁

其中, r_1 和 r_2 是上述两条规则在动物识别系统中的规则编号。

假设已知有以下事实:

动物有羽毛, 动物善飞

求满足以上事实的动物是何种动物。

解: 由于已知事实"动物有羽毛", 即 r_1 的前提条件满足, 因此 r_1 可用, 承认的 r_1 结论, 即推出新的事实"动物是鸟"。此时, r_2 的两个前提条件均满足, 即 r_2 的前提条件满足, 因此 r_2 可用, 承认的 r_2 结论, 即推出新的事实"动物是信天翁"。

2.3.4 产生式系统应用举例

例1:设有以下两条规则

r_1 : IF 动物有羽毛 THEN 动物是鸟

r_2 : IF 动物是鸟 AND 动物善飞 THEN 动物是信天翁

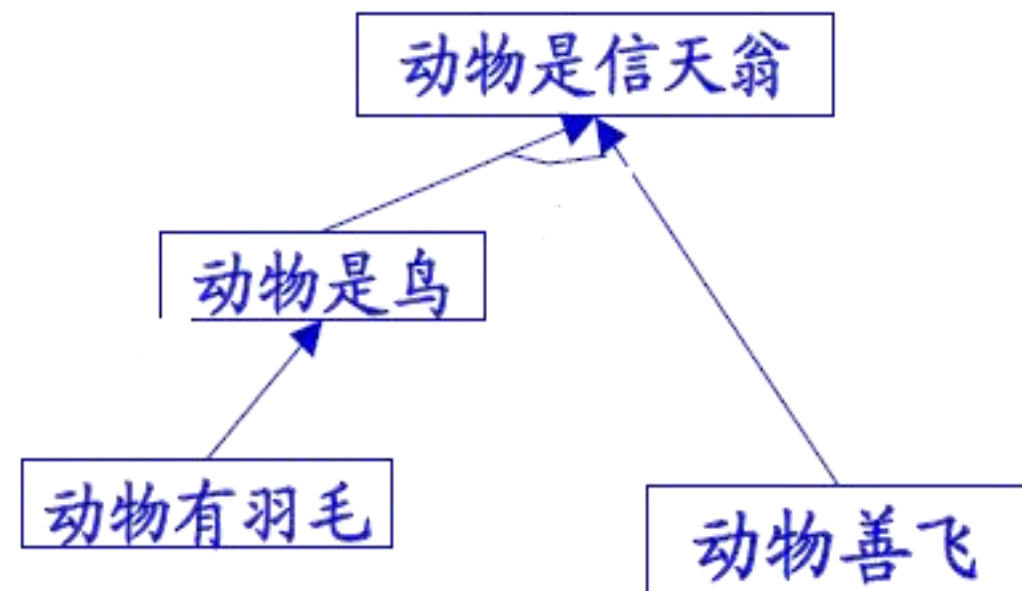
其中, r_1 和 r_2 是上述两条规则在动物识别系统中的规则编号。

假设已知有以下事实:

动物有羽毛, 动物善飞

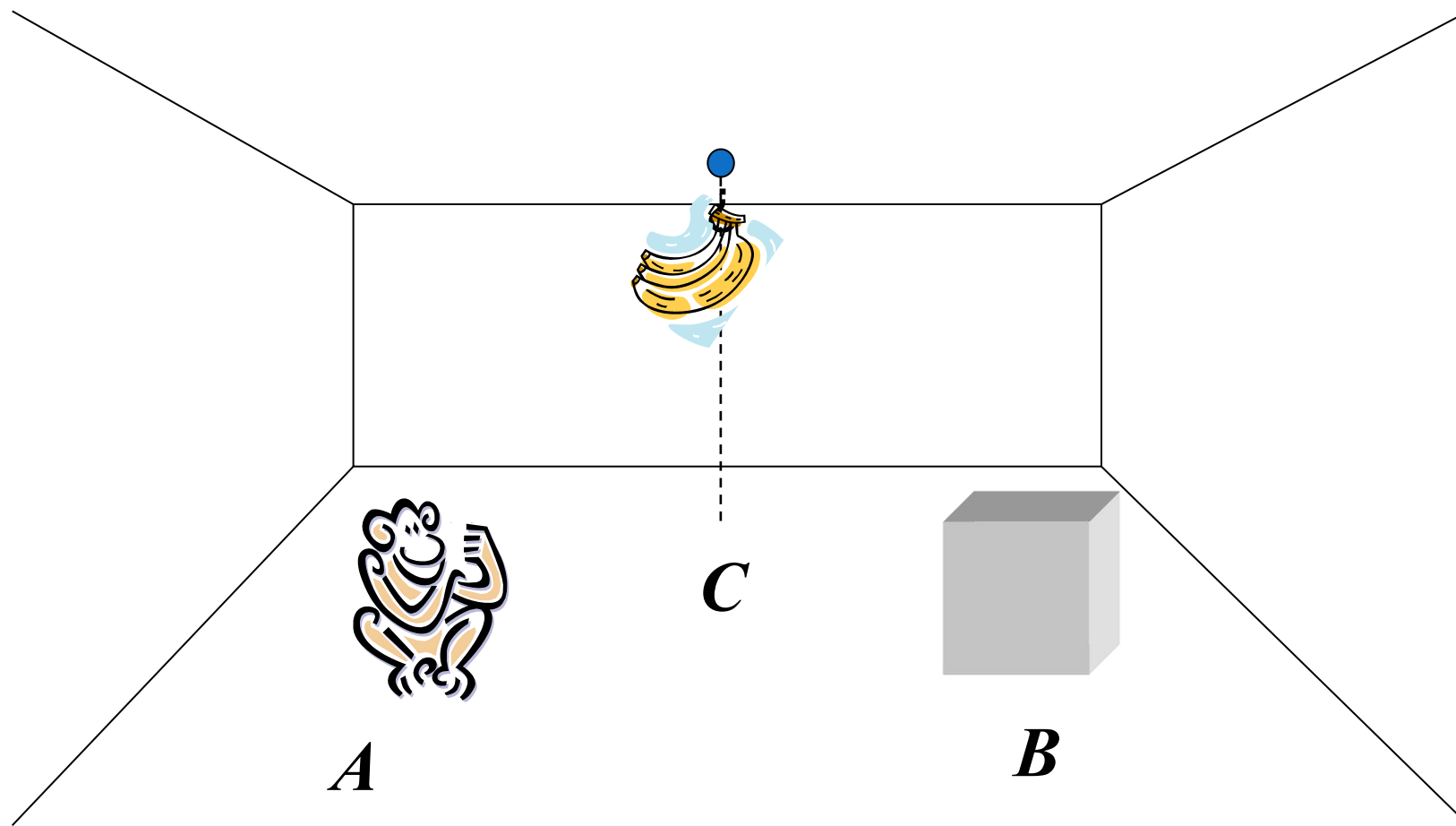
求满足以上事实的动物是何种动物。

解: 由于已知事实"动物有羽毛", 即 r_1 的前提条件满足, 因此 r_1 可用, 承认的 r_1 结论, 即推出新的事实"动物是鸟"。此时, r_2 的两个前提条件均满足, 即 r_2 的前提条件满足, 因此 r_2 可用, 承认的 r_2 结论, 即推出新的事实"动物是信天翁"。



2.3.4 产生式系统应用举例

猴子摘香蕉问题



2.3.4 产生式系统应用举例

求解: (1) 综合数据库: (M, B, Box, On, H)

M: 猴子的位置

B: 香蕉的位置

Box: 箱子的位置

On=0: 猴子在地板上

On=1: 猴子在箱子上

H=0: 猴子没有抓到香蕉

H=1: 猴子抓到了香蕉

(2) 初始状态: $(a, c, b, 0, 0)$

2.3.4 产生式系统应用举例

(3) 结束状态：

(c, c, c, 1, 1)

(4) 规则集：

r_1 : IF (x, y, z, 0, 0) THEN (w, y, z, 0, 0) (猴子移动)

r_2 : IF (x, y, x, 0, 0) THEN (z, y, z, 0, 0) (猴子推箱子)

r_3 : IF (x, y, x, 0, 0) THEN (x, y, x, 1, 0) (猴子爬箱子)

r_4 : IF (x, y, x, 1, 0) THEN (x, y, x, 0, 0) (猴子下箱子)

r_5 : IF (x, x, x, 1, 0) THEN (x, x, x, 1, 1) (猴子抓香蕉)

其中, x, y, z, w 为变量。

2.3.4 产生式系统应用举例

解答： 根据具体问题可将规则具体为：

r1: IF (a, c, b, 0, 0) THEN (b, c, b, 0, 0)

r2: IF (b, c, b, 0, 0) THEN (c, c, c, 0, 0)

~~r3: IF (b, c, b, 0, 0) THEN (b, c, b, 1, 0)~~

r3: IF (c, c, c, 0, 0) THEN (c, c, c, 1, 0)

~~r4: IF (b, c, b, 1, 0) THEN (b, c, b, 0, 0)~~

r5: IF (c, c, c, 1, 0) THEN (c, c, c, 1, 1)

在已知事实下, $r1 \rightarrow r2 \rightarrow r3 \rightarrow r5$, 可得到香蕉

2.3.4 产生式系统应用举例

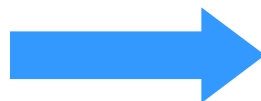
例2: 传教士与野人问题。N个传教士，N个野人，一条船，可同时乘坐k个人，要求在任何时刻，在河的两岸，传教士和野人同时存在时，传教士的人数不能少于野人的人数。

问：如何过河？(以 $N=3$ ， $k=2$ 为例求解。)

描述:

初始状态

	L	R
m	3	0
c	3	0
b	1	0



目标状态

	L	R
m	0	3
c	0	3
b	0	1

2.3.4 产生式系统应用举例

(1) 综合数据库: (m, c, b)

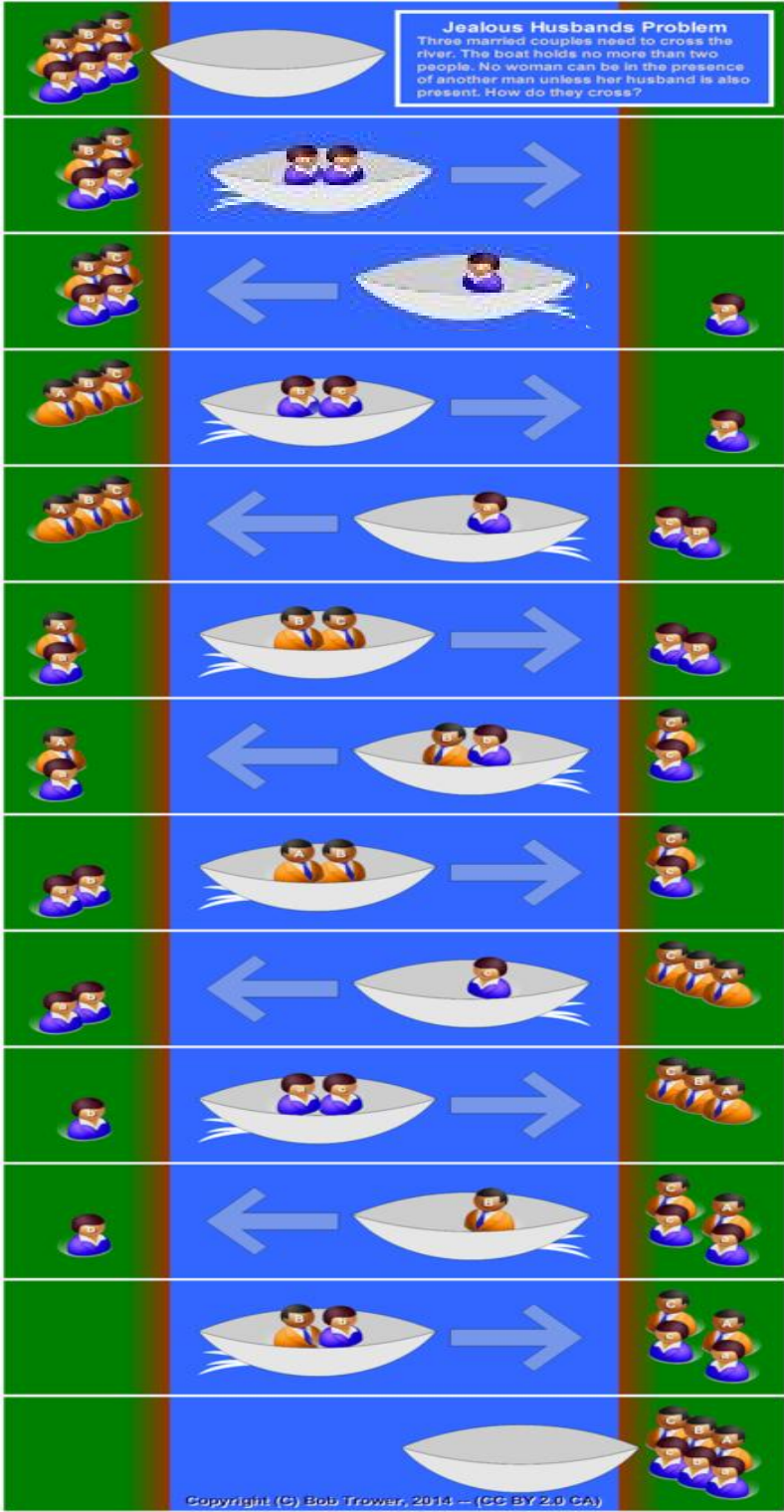
其中, $0 \leq m \leq 3, 0 \leq c \leq 3, b \in \{0, 1\}$

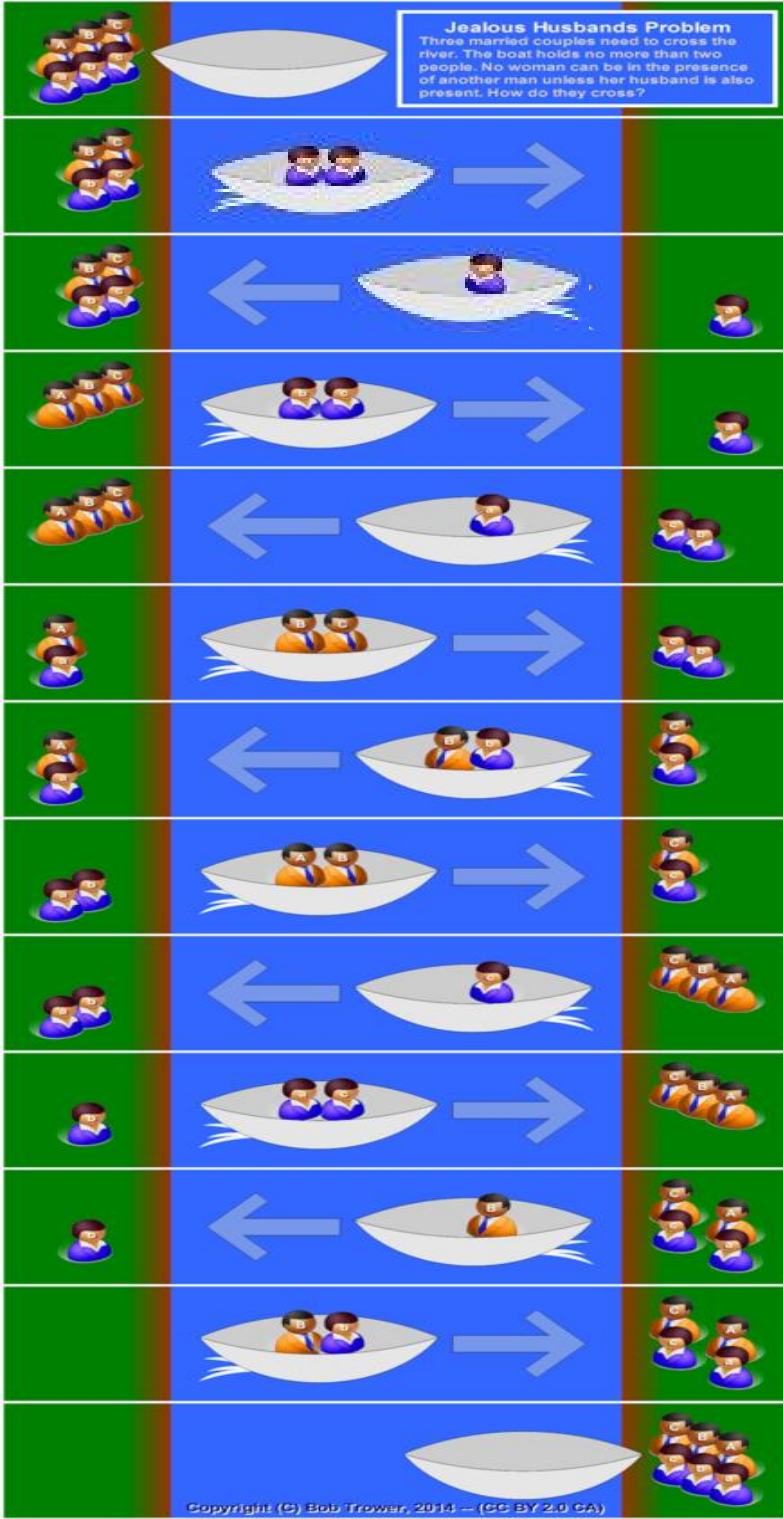
(2) 初始状态: $(3, 3, 1)$

(3) 目标状态: $(0, 0, 0)$

(4) 规则集

- | | |
|---|---|
| $r1: \text{ IF } (m, c, 1) \text{ THEN } (m-1, c, 0)$ | $r6: \text{ IF } (m, c, 0) \text{ THEN } (m+1, c, 1)$ |
| $r2: \text{ IF } (m, c, 1) \text{ THEN } (m, c-1, 0)$ | $r7: \text{ IF } (m, c, 0) \text{ THEN } (m, c+1, 1)$ |
| $r3: \text{ IF } (m, c, 1) \text{ THEN } (m-1, c-1, 0)$ | $r8: \text{ IF } (m, c, 0) \text{ THEN } (m+1, c+1, 1)$ |
| $r4: \text{ IF } (m, c, 1) \text{ THEN } (m-2, c, 0)$ | $r9: \text{ IF } (m, c, 0) \text{ THEN } (m+2, c, 1)$ |
| $r5: \text{ IF } (m, c, 1) \text{ THEN } (m, c-2, 0)$ | $r10: \text{ IF } (m, c, 0) \text{ THEN } (m, c+2, 1)$ |





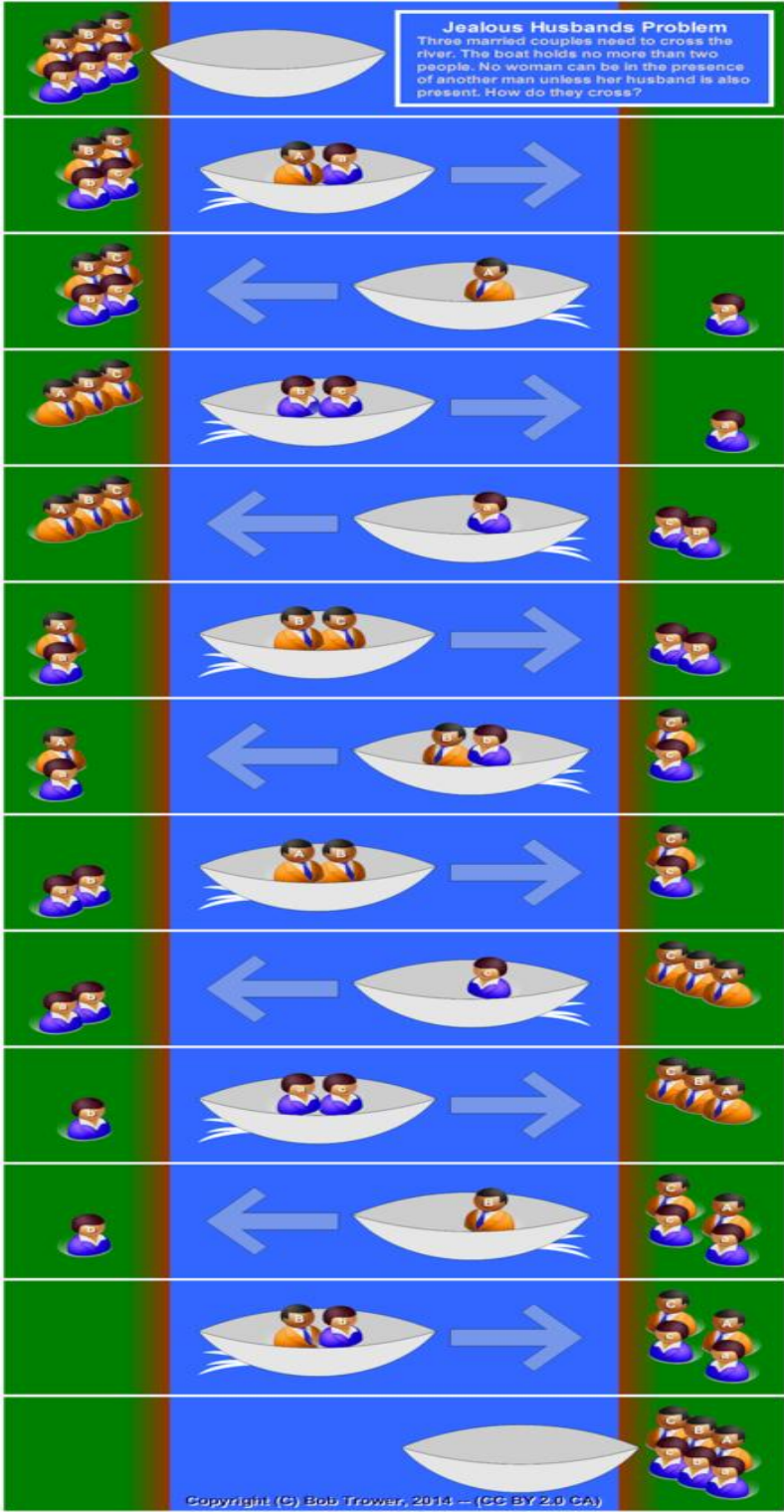
解答:

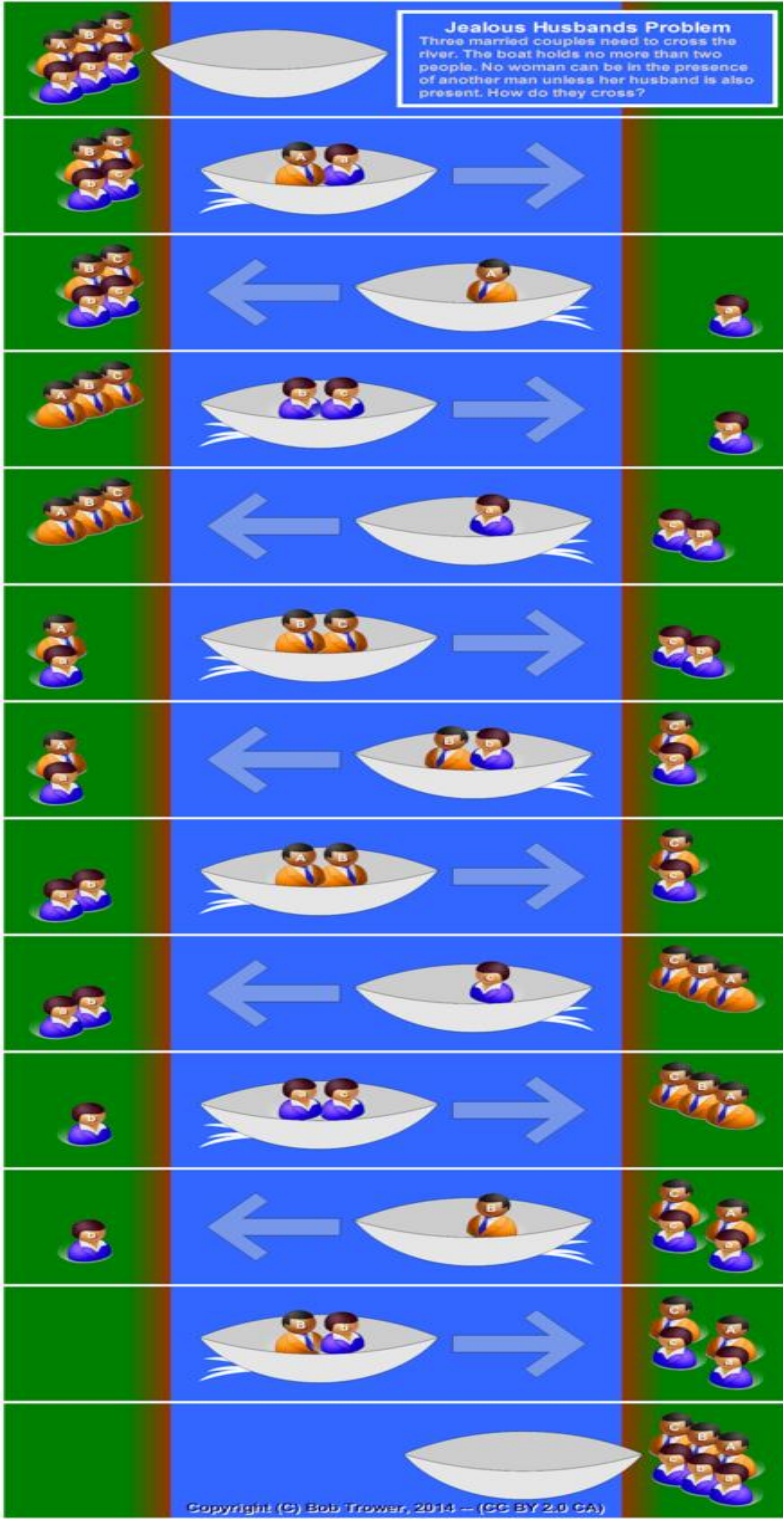
根据具体问题可将规则具体为:

- r5: IF (3, 3, 1) THEN (3, 1, 0) (船运**2个c**到右岸)
- r7: IF (3, 1, 0) THEN (3, 2, 1) (船运**1个c**到左岸)
- r5: IF (3, 2, 1) THEN (3, 0, 0) (船运**2个c**到右岸)
- r7: IF (3, 0, 0) THEN (3, 1, 1) (船运**1个c**到左岸)
- r4: IF (3, 1, 1) THEN (1, 1, 0) (船运**2个m**到右岸)
- r8: IF (1, 1, 0) THEN (2, 2, 1) (船运**1个m和1个c**到左岸)
- r4: IF (2, 2, 1) THEN (0, 2, 0) (船运**2个m**到右岸)
- r7: IF (0, 2, 0) THEN (0, 3, 1) (船运**1个c**到左岸)
- r5: IF (0, 3, 1) THEN (0, 1, 0) (船运**2个c**到右岸)
- r7: IF (0, 1, 0) THEN (0, 2, 1) (船运**1个c**到左岸)
- r5: IF (0, 2, 1) THEN (0, 0, 0) (船运**2个c**到右岸)

在已知事实下:

r5→r7→r5→r7→r4→r8→r4→r7→r5→r7→r5 ,可顺利过河。





Jealous Husbands Problem
Three married couples need to cross the river. The boat holds no more than two people. No woman can be in the presence of another man unless her husband is also present. How do they cross?

解答：

根据具体问题可将规则具体为：

- r3: IF (3, 3, 1) THEN (2, 2, 0) (船运1个m和1个c到右岸)
- r6: IF (2, 2, 0) THEN (3, 2, 1) (船运1个m到左岸)
- r5: IF (3, 2, 1) THEN (3, 0, 0) (船运2个c到右岸)
- r7: IF (3, 0, 0) THEN (3, 1, 1) (船运1个c到左岸)
- r4: IF (3, 1, 1) THEN (1, 1, 0) (船运2个m到右岸)
- r8: IF (1, 1, 0) THEN (2, 2, 1) (船运1个m和1个c到左岸)
- r4: IF (2, 2, 1) THEN (0, 2, 0) (船运2个m到右岸)
- r7: IF (0, 2, 0) THEN (0, 3, 1) (船运1个c到左岸)
- r5: IF (0, 3, 1) THEN (0, 1, 0) (船运2个c到右岸)
- r7: IF (0, 1, 0) THEN (0, 2, 1) (船运1个c到左岸)
- r5: IF (0, 2, 1) THEN (0, 0, 0) (船运2个c到右岸)

在已知事实下：

r3→r6→r5→r7→r4→r8→r4→r7→r5→r7→r5 ,可顺利过河。

2.3.5 产生式系统的特点

□主要优点

- 自然性: “如果……, 则……”,符合人类习惯
- 模块性: 基本知识单元, 静态关系, 正确性独立, 无需调用
- 有效性: 既可以表示确定性和非确定性知识皆可表示

2.3.5 产生式系统的特点

□主要优点

- 自然性: “如果……, 则……”,符合人类习惯
- 模块性: 基本知识单元, 静态关系, 正确性独立, 无需调用
- 有效性: 既可以表示确定性和非确定性知识皆可表示

□主要缺点

- 效率较低: 受综合数据局的约束,反复进行"匹配—冲突消解—执行"。
- 不便于表示结构性知识: 知识具有一致格式, 且规则之间不能相互调用, 难以表示结构化知识。

目 录

- 01 知识与知识表示
- 02 一阶谓词逻辑表示法
- 03 产生式表示法
- 04 语义网络表示法
- 05 框架表示法

2.4 语义网络表示法

2.4.1 概 述

- 1968年 R.Quillian 在研究人类联想记忆时提出，后把它用作知识表示。
- 逻辑和产生式常用于表示有关领域中各个不同状态间的关系。
- 语义网络和产生式、一阶谓词逻辑有相对应的表示能力。

2.4 语义网络表示法

□ 概述

- 1968年 R.Quillian 在研究人类联想记忆时提出，后把它用作知识表示。
- 逻辑和产生式常用于表示有关领域中各个不同状态间的关系。
- 语义网络和产生式、一阶谓词逻辑有相对应的表示能力。

□ 语义网络

- 通过概念及语义关系来表示知识的一种网络图，它是一个带标注的有向图。
- 图中的各个节点表示各种概念、事物、对象、行为、状态等；
- 图中的有向弧表示节点间的联系或关系。

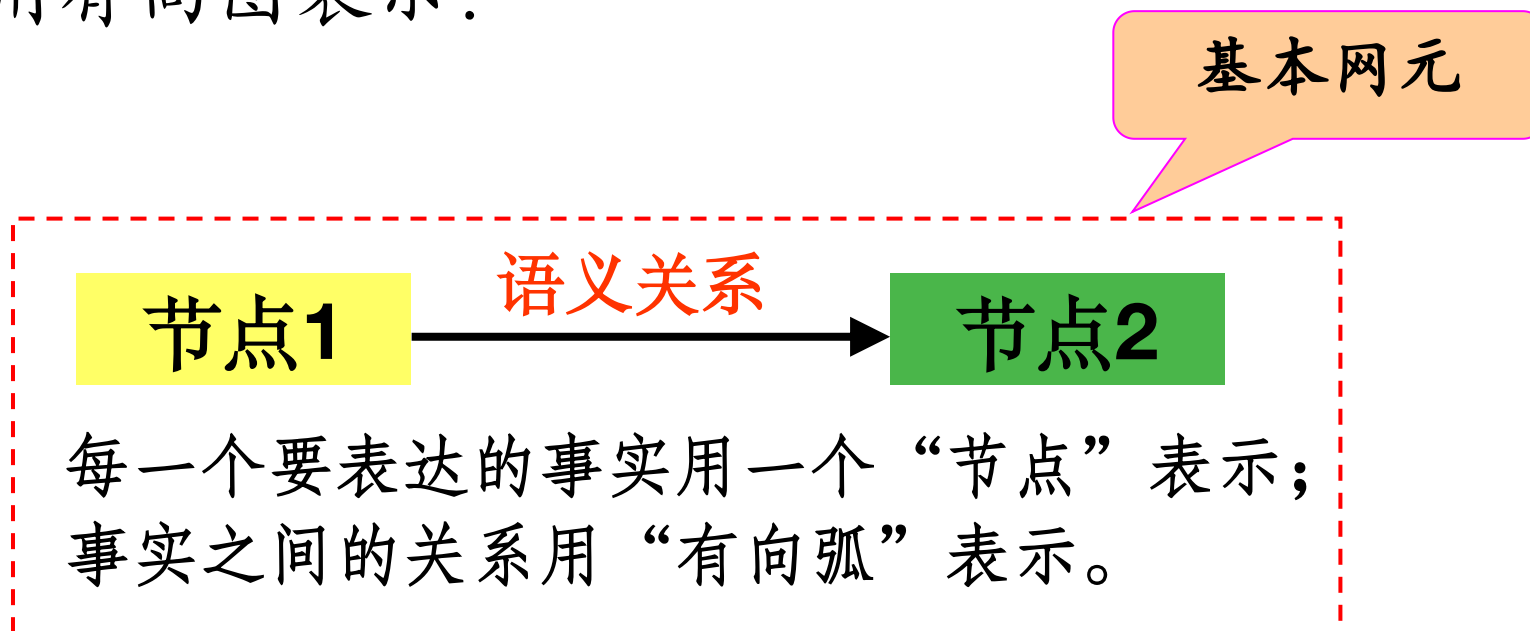
2.4.1 语义网络表示法概述

□ 语义网络的基本表示

一般由一些最基本的语义单元组成。这些最基本的语义单元被称为**语义基元**。可用如下三元组来表示：

(节点1, 弧, 节点2)

也可用有向图表示：



2.4.1 语义网络表示法概述

□ 语义网络表示实例

例1: “小李和小王是朋友”

语义网络表示:



一阶谓词逻辑表示:

定义谓词: **Friend(x, y)**.

定义个体词: **Li**:小李; **Wang**:小王

表示为: **Friend(Li, Wang)**

产生式表示: **(Friend, Li, Wang)**

2.4.1 语义网络表示法概述

把多个基本网元用相应的语义联系关联在一起时，就可得到一个语义网络。

语义网络中的节点还可以是一个语义子网络，所以，语义网络实质上是一种多层次的嵌套结构。

2.4.2 基本语义关系

(1)实例关系: ISA

体现的是“**具体与抽象**”的概念，含义为“**是一个**”，表示一个事物是另一个事物的一个实例。



(2)分类关系: AKO

亦称泛化关系，体现的是“**子类与超类**”的概念，含义为“**是一种**”，表示一个事物是另一个事物的一种类型。



(3)成员关系: A-Member-of

体现的是“**个体与集体**”的关系，含义为“**是一员**”，表示一个事物是另一个事物的一个成员。

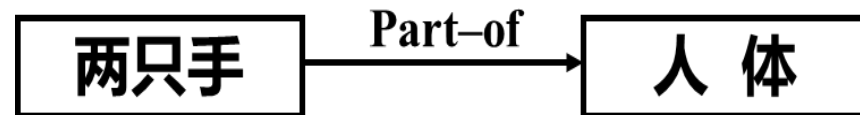


一个最主要的特征是**属性的继承性**，处在具体层的节点可以继承所有抽象层节点的所有属性。

2.4.2 基本语义关系

(4) 聚类关系

亦称包含关系。指具有组织或结构特征的"部分与整体"之间的关系。常用的包含关系是：



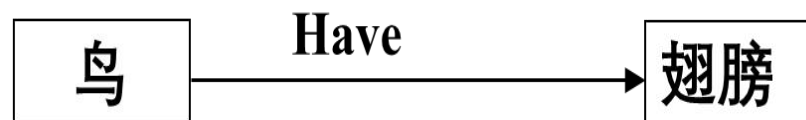
Part-of: 含义为"是一部分", 表示一个事物是另一个事物的一部分。

聚类关系与实例、分类、成员关系的主要区别聚类关系一般不具备属性的继承性。如上例, 手不一定具有人的各种属性

(5) 属性关系

指事物和其属性之间的关系。常用的有：

Have: 含义为"有", 表示一个结点具有另一个结点所描述的属性



Can: 含义为"能"、"会", 表示一个结点能做另一个结点的事情

2.4.2 基本语义关系

(6)时间关系

指不同事件在其发生时间方面的先后次序关系。

常用的时间关系有：

Before: 含义为"在前"

After: 含义为"在后"



(7)位置关系

指不同事物在位置方面的关系.常用的有：

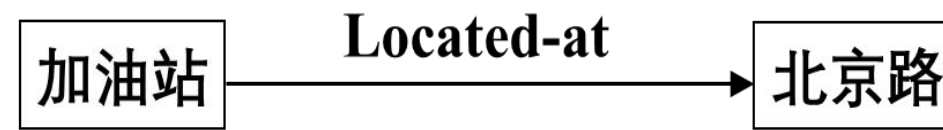
Located-on: 含义为"在...上面"

Located-under: 含义为"在...下面"

Located-at: 含义为"在..."

Located-inside: 含义为"在...内"

Located-outside: 含义为"在...外"



2.4.2 基本语义关系

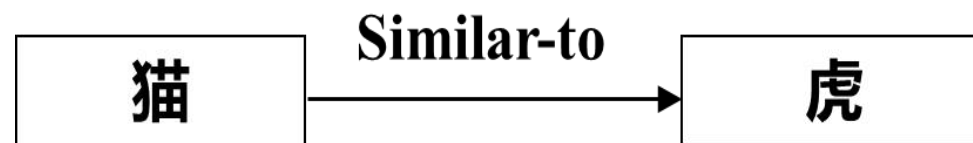
(8)相近关系

指不同事物在形状、内容等方面相似或接近。

常用的相近关系有：

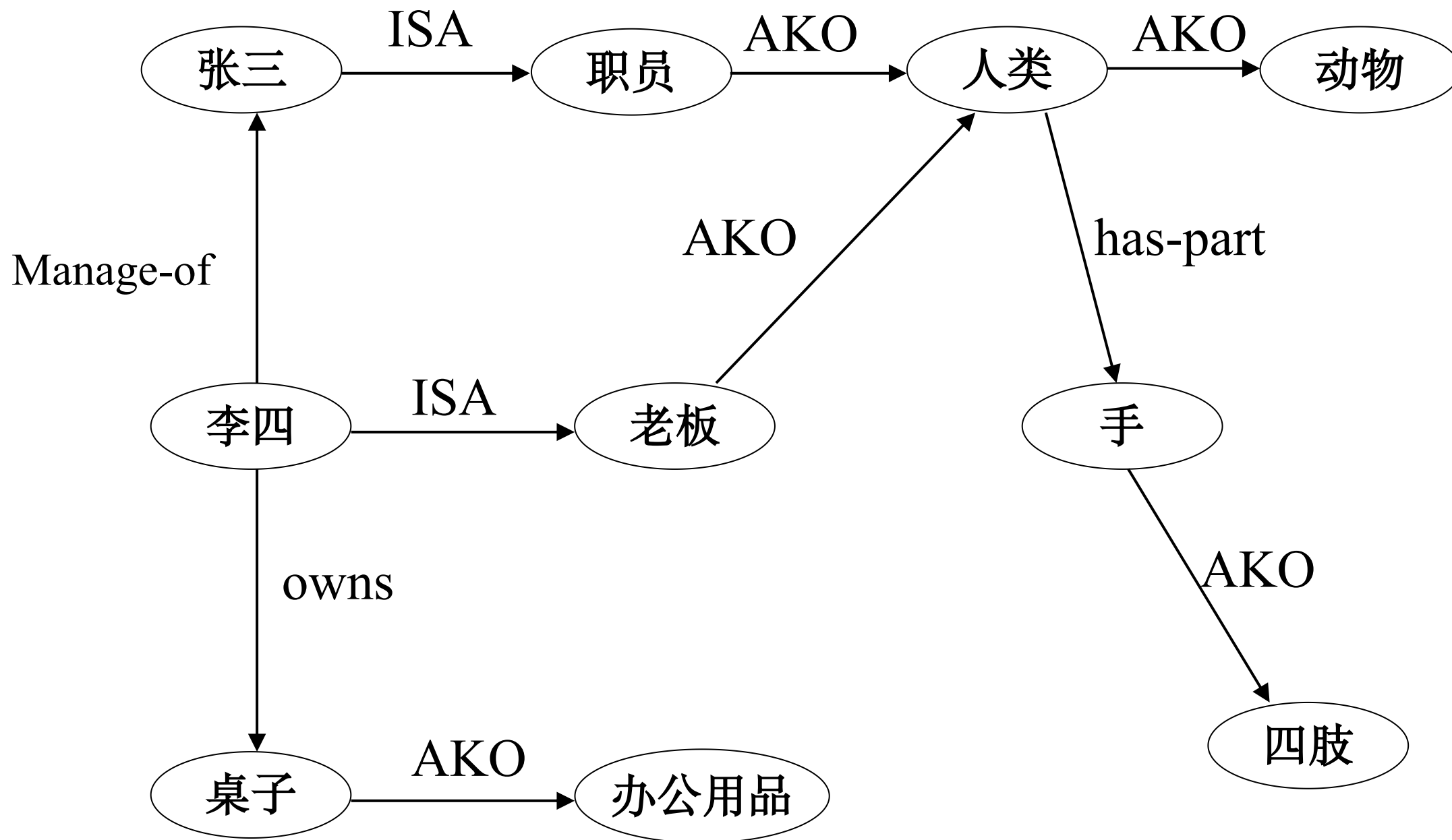
Similar-to: 含义为"相似"

Near-to: 含义为"接近"



2.4.2 基本语义关系

例2:



2.4.3 事物和概念的表达

□ 表示一元关系

➤ 一元关系

指可以用一元谓词 $P(x)$ 表示的关系。谓词 P 说明实体的性质、属性等。描述的是一些最简单、最直观的事物或概念，常用：“是”、“有”、“会”、“能”等语义关系来说明。如，“雪是白的”。

➤ 一元关系的描述

应该说，语义网络表示的是二元关系。如何用它来描述一元关系？
结点1表示实体，结点2表示实体的性质或属性等，弧表示语义关系。

2.4.3 事物和概念的表达

□ 表示一元关系

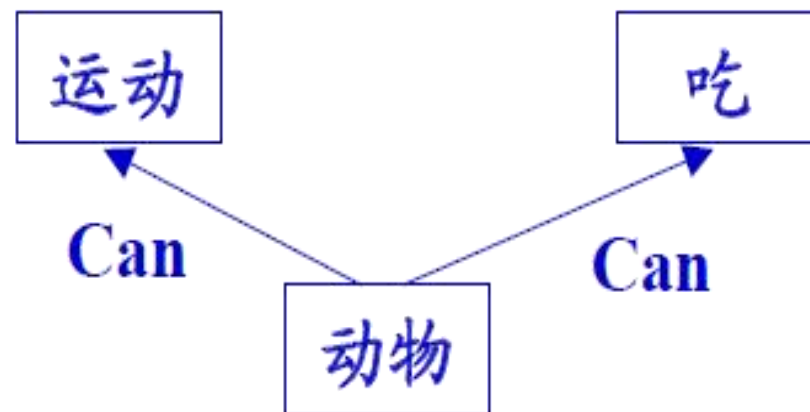
➤ 一元关系

指可以用一元谓词 $P(x)$ 表示的关系。谓词 P 说明实体的性质、属性等。描述的是一些最简单、最直观的事物或概念，常用：“是”、“有”、“会”、“能”等语义关系来说明。如，“雪是白的”。

➤ 一元关系的描述

应该说，语义网络表示的是二元关系。如何用它来描述一元关系？
结点1表示实体，结点2表示实体的性质或属性等，弧表示语义关系。

例：用语义网络表示“动物能运动、会吃”。



2.4.3 事物和概念的表达

□ 表示二元关系

➤ 二元关系

指可用二元谓词 $P(x, y)$ 表示的关系。其中， x, y 为实体， P 为实体之间的关系。

➤ 二元关系的描述

单个二元关系可直接用一个基本网元来表示。

复杂关系，可通过一些相对独立的二元或一元关系的组合来实现。

2.4.3 事物和概念的表达

□ 表示二元关系

➤ 二元关系

指可用二元谓词 $P(x, y)$ 表示的关系。其中， x, y 为实体， P 为实体之间的关系。

➤ 二元关系的描述

单个二元关系可直接用一个基本网元来表示。

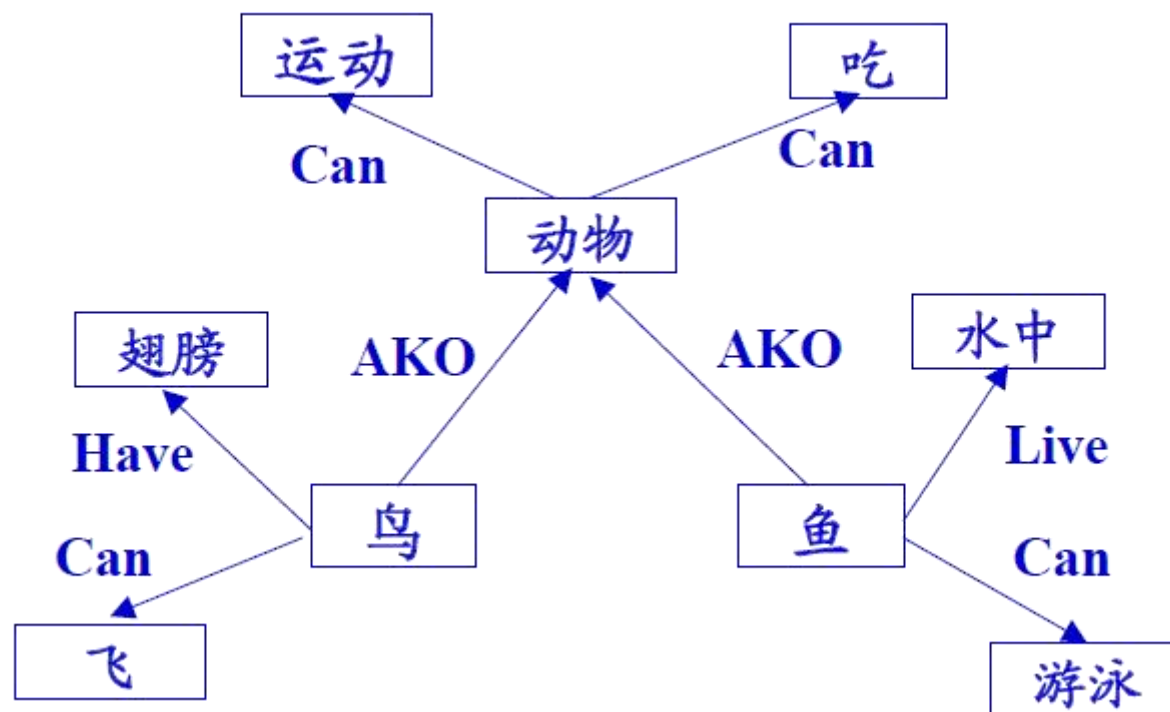
复杂关系，可通过一些相对独立的二元或一元关系的组合来实现。

例：用语义网络表示：

动物能运动、会吃。

鸟是一种动物，鸟有翅膀、会飞。

鱼是一种动物，鱼生活在水中、会游泳。



2.4.3 事物和概念的表达

例：用语义网络表示：

王强是理想公司的经理；

理想公司在中关村；

王强28岁。

2.4.3 事物和概念的表达

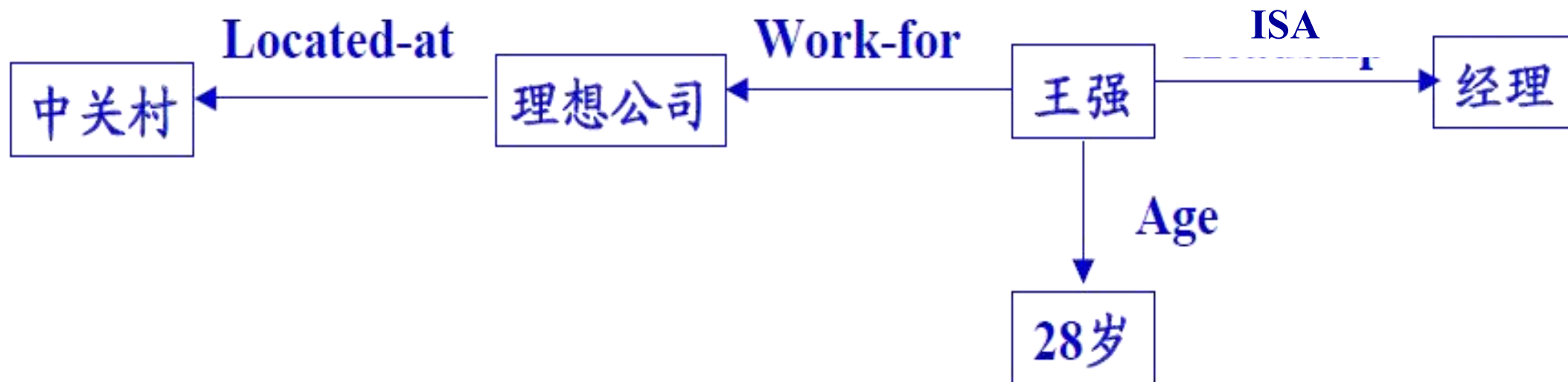
□ 实例

例：用语义网络表示：

王强是理想公司的经理；

理想公司在中关村；

王强28岁。



2.4.3 事物和概念的表达

例：用语义网络表示：

李新的笔记本的牌子是"联想"、颜色黑灰。

王红的笔记本的牌子是"方正"、颜色粉红。

解：李新和王红的笔记本均属于具体概念，可增加"笔记本"这个抽象概念。

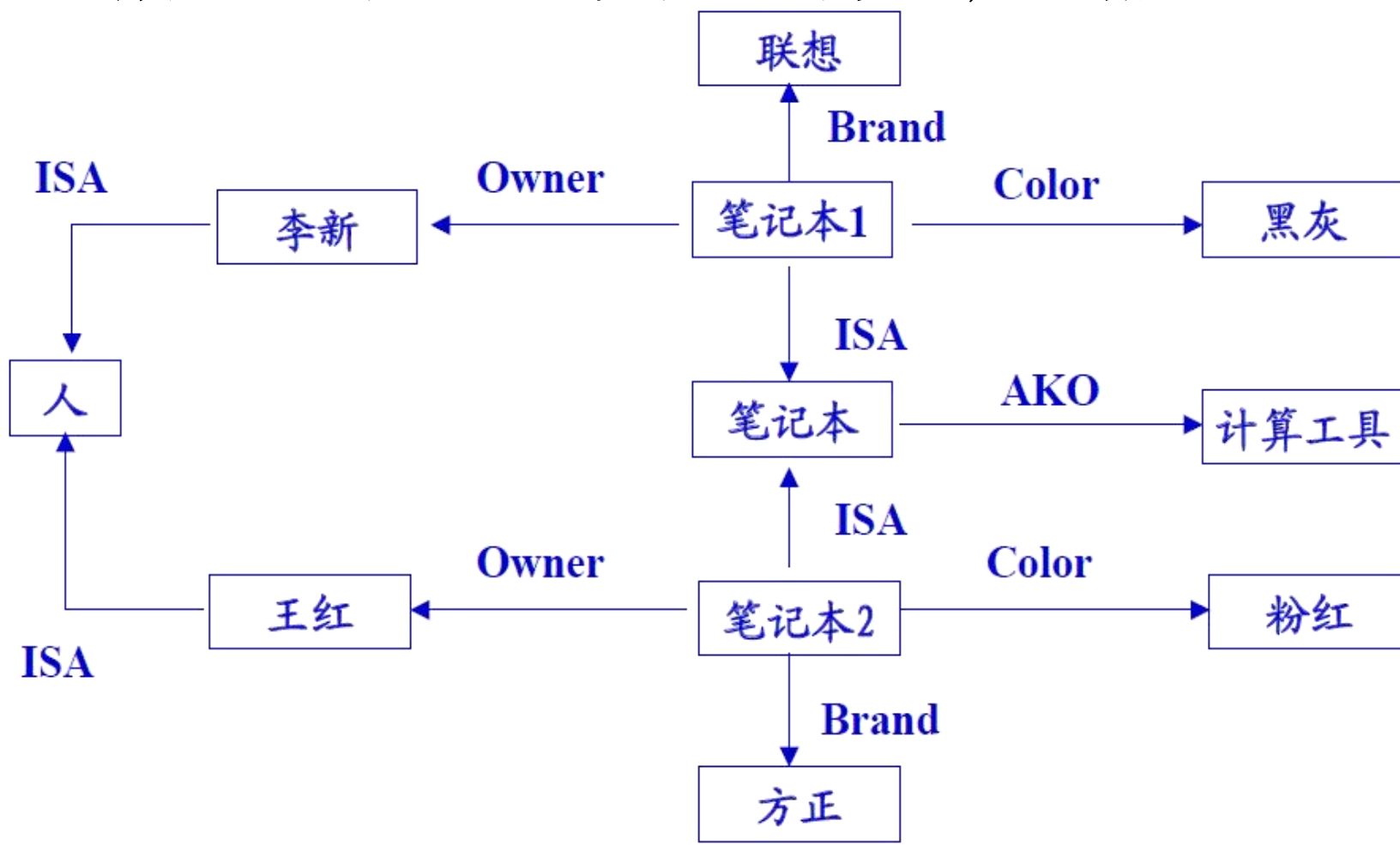
2.4.3 事物和概念的表达

例：用语义网络表示：

李新的笔记本的牌子是"联想"、颜色黑灰。

王红的笔记本的牌子是"方正"、颜色粉红。

解：李新和王红的笔记本均属于具体概念，可增加"笔记本"这个抽象概念。



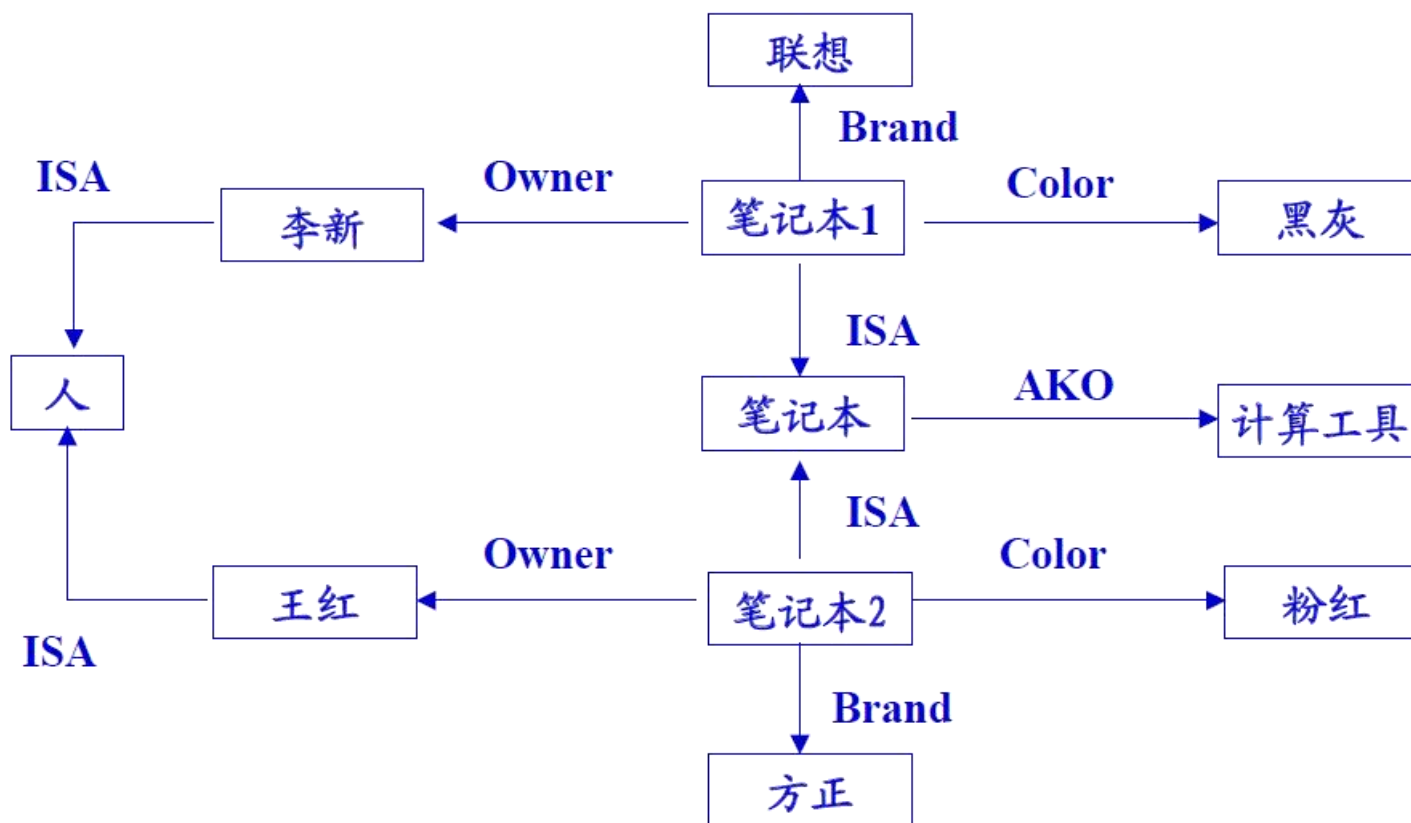
2.4.3 事物和概念的表达

例：用语义网络表示：

李新的笔记本的牌子是"联想"、颜色黑灰。

王红的笔记本的牌子是"方正"、颜色粉红。

解：李新和王红的笔记本均属于具体概念，可增加"笔记本"这个抽象概念。



2.4.3 事物和概念的表达

□ 表示多元关系

➤ 多元关系

可用多元谓词 $P(x_1, x_2, \dots)$ 表示的关系。其中, $x_1, x_2, \dots$ 为实体, 谓词 P 说明这些实体之间的关系。

➤ 多元关系的表示

用语义网络表示多元关系时, 可把它转化为一个或多个二元关系的组合, 把这种多元关系表示出来。

2.4.4 情况和动作的表示

□ 情况的表示

➤ **表示方法**: 西蒙提出了增加情况和动作结点的描述方法。

例: 用语义网络表示:

“小燕子这只燕子从春天到秋天占有一个巢”

对上述问题, 可以把占有用一条弧来表示, 但在这种表示方法下, 占有关系就无法表示了

2.4.4 情况和动作的表示

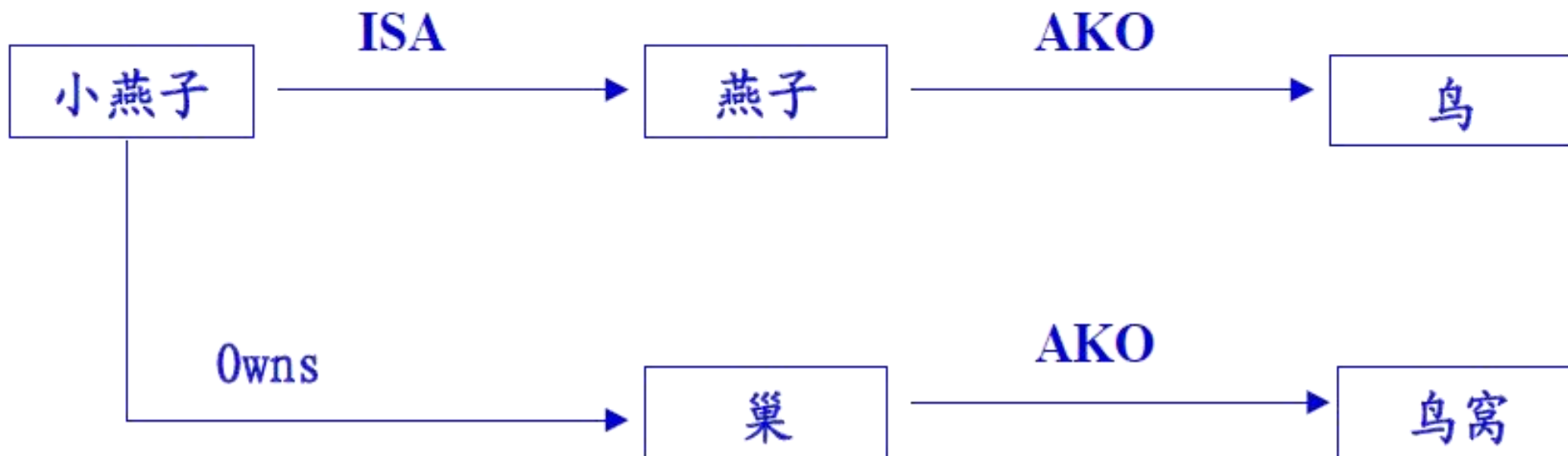
□ 情况的表示

➤ **表示方法**: 西蒙提出了增加情况和动作结点的描述方法。

例: 用语义网络表示:

“小燕子这只燕子从春天到秋天占有一个巢”

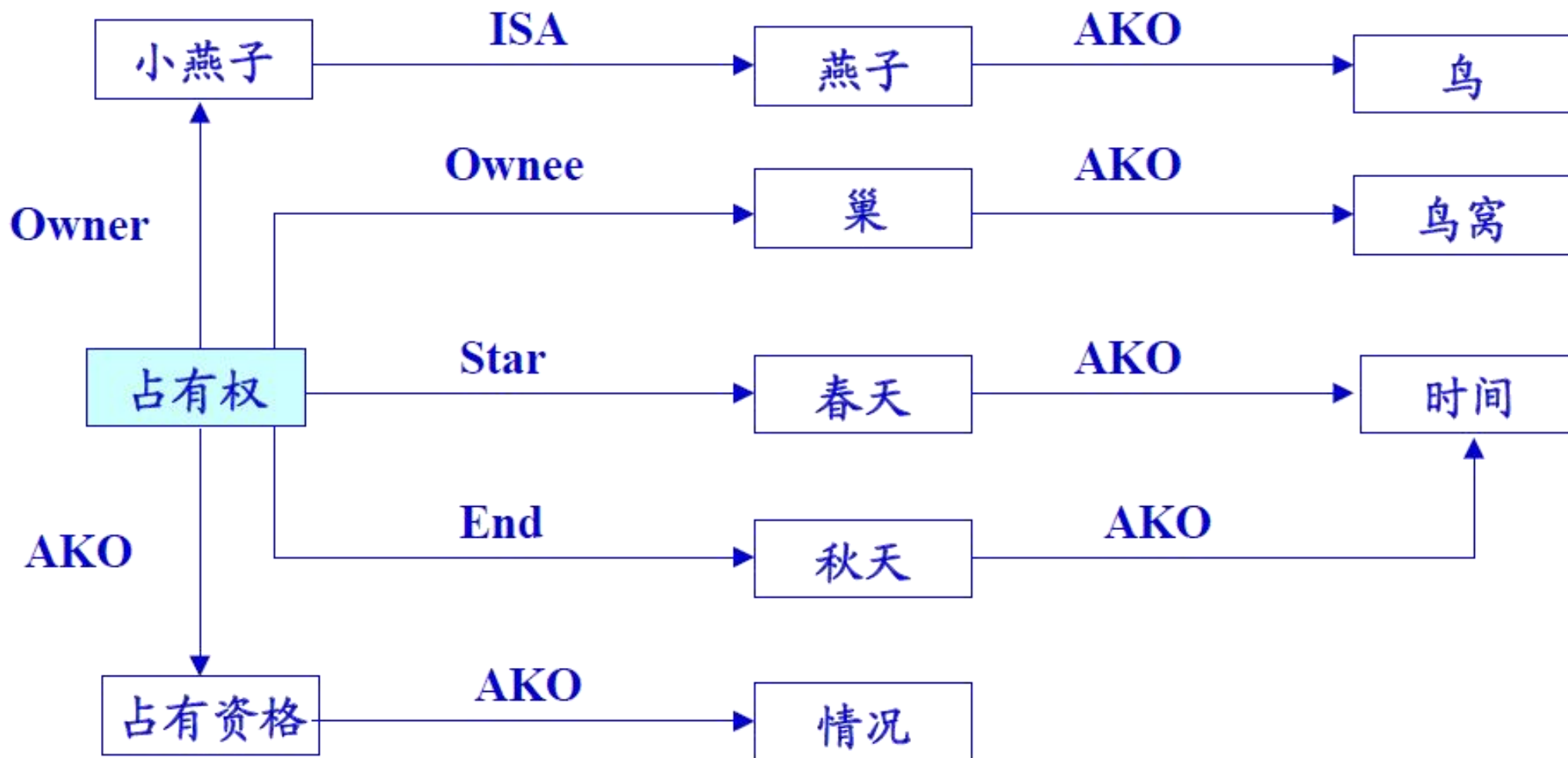
对上述问题, 可以把占有用一条弧来表示, 但在这种表示方法下, 占有关系就无法表示了



2.4.4 情况和动作的表示

□ 情况的表示

解：需要设立一个占有权结点，表示占有物和占有时间等。



2.4.4 情况和动作的表示

□ 事件和动作的表示

用这种方法表示事件或动作时，需要设立一个事件节点或动作结点。其中，事件节点由一些向外引出的弧来指出事件行为及发出者与接受者。动作结点由一些向外引出的弧来指出动作的主体与客体。

2.4.4 情况和动作的表示

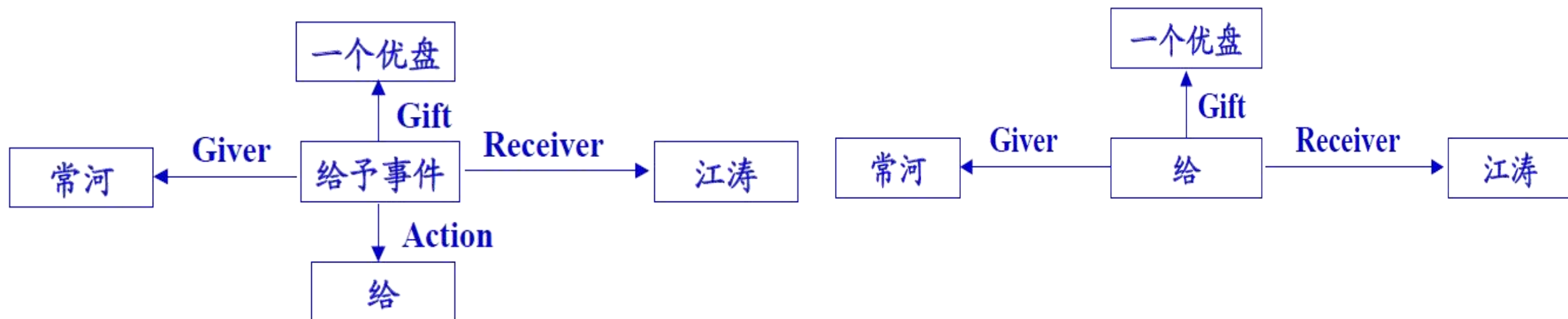
□ 事件和动作的表示

用这种方法表示事件或动作时，需要设立一个**事件节点**或**动作结点**。其中，事件节点由一些向外引出的弧来指出**事件行为及发出者与接受者**。动作结点由一些向外引出的弧来指出**动作的主体与客体**。

例：用于语义网络表示："常河给江涛一个优盘"

解：

用**动作结点**节点表示



2.4.5 基于语义网络的推理

➤ **推理类别**：语义网络的推理过程主要有两种，一种是继承，另一种是匹配。

➤ 继承的概念

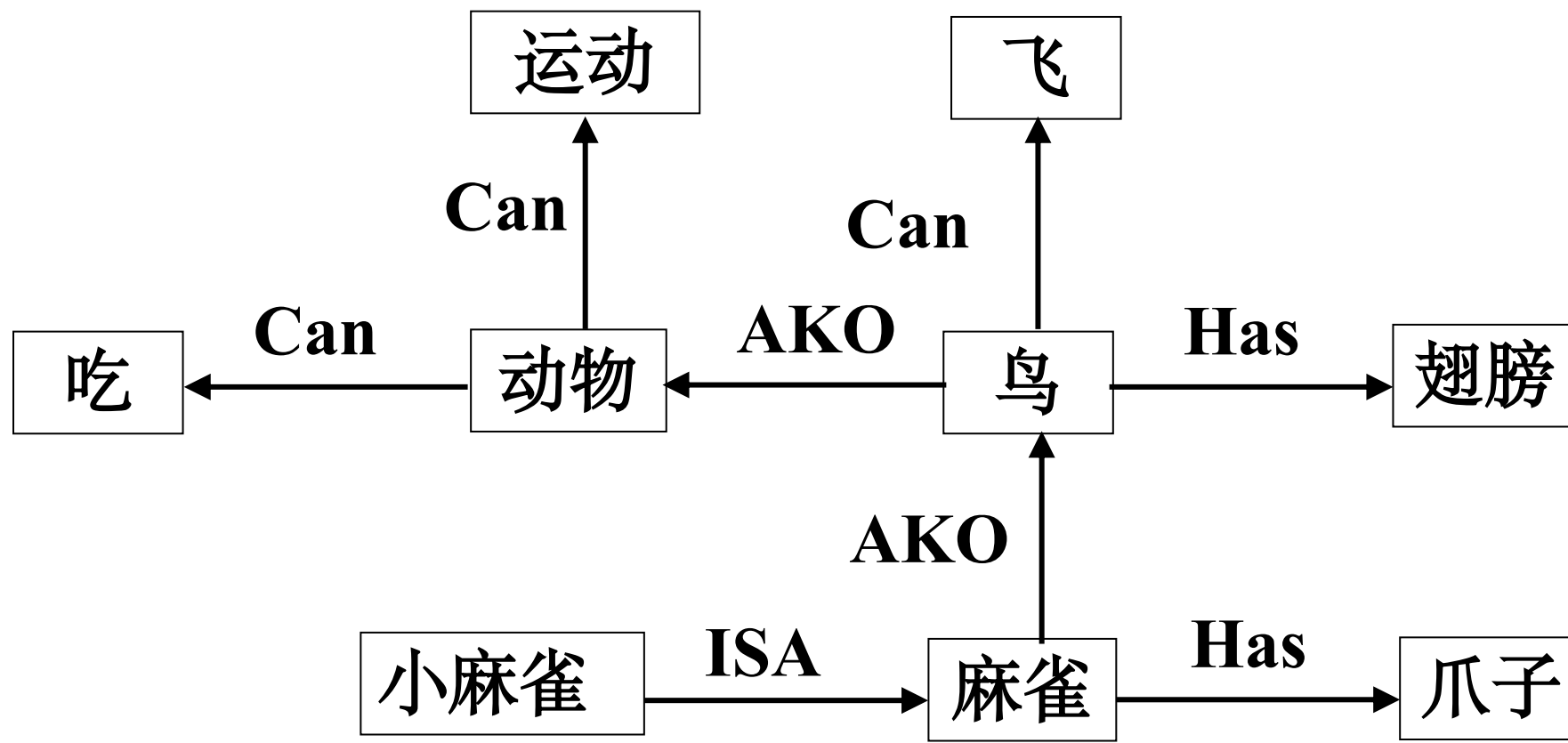
是指把对事物的描述从抽象结点传递到实例结点。通过继承可以得到所需结点的一些属性值，它通常是沿着ISA、AKO等继承弧进行的。

➤ 继承的一般过程

- ❖ 建立一个结点表，用来存放待求解结点和所有以ISA、AKO等继承弧与此结点相连的那些结点。初始情况下，表中只有待求解结点。
- ❖ 检查表中的第一个结点是否有继承弧。如果有，就把该弧所指的所有结点放入结点表的末尾，记录这些结点的所有属性，并从结点表中删除第一个结点。如果没有继承弧，仅从结点表中删除第一个结点。
- ❖ 重复(2)，直到结点表为空。此时，记录下来的所有属性都是待求解结点继承来的属性。

2.4.5 基于语义网络的推理

例： 用语义网络表示：
动物能运动、会吃。
鸟是一种动物，鸟有翅膀、会飞。
麻雀有爪子，麻雀是一种鸟。
小麻雀是一只麻雀。



2.4.5 基于语义网络的推理

➤ 匹配的概念

是指在知识库的语义网络中寻找与待求解问题相符的语义网络模式。

➤ 匹配的过程

- ❖ 根据待求解问题的要求构造一个网络片断，该网络片断中有些结点或孤的标识是空的，称为询问处，它反映的是待求解的问题。
- ❖ 根据该语义片断到知识库中去寻找所需要的信息。
- ❖ 当待求解问题的网络片断与知识库中的某语义网络片断相匹配时，则与询问处相匹配的事实就是问题的解。

2.4.5 基于语义网络的推理

例：设在语义网络系统的知识库中存在以下事实的语义网络：

“哈尔滨工业大学是一所学校，位于哈尔滨市，成立于1920年。”

假若要求解的问题是：哈尔滨工业大学位于哪个城市？

如何利用语义网络进行推理求解？

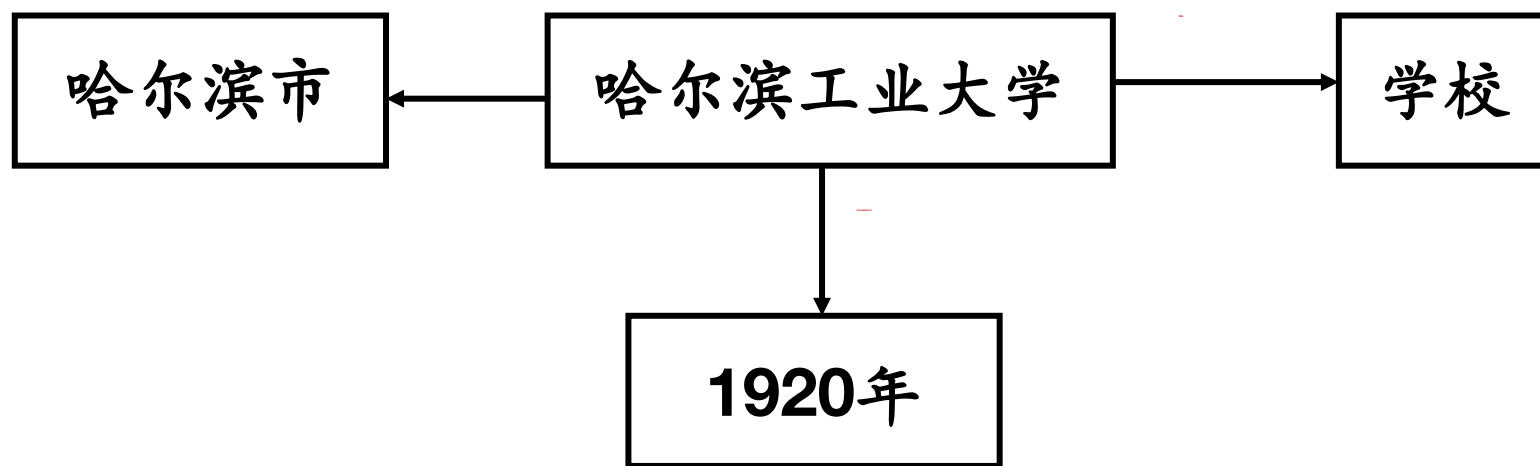
2.4.5 基于语义网络的推理

例：设在语义网络系统的知识库中存在以下事实的语义网络：

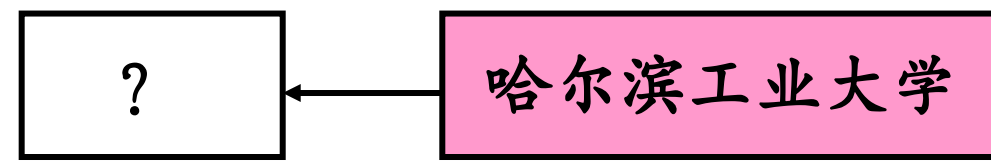
“哈尔滨工业大学是一所学校，位于哈尔滨市，成立于1920年。”

假若要求解的问题是：哈尔滨工业大学位于哪个城市？

如何利用语义网络进行推理求解？



知识库中关于哈尔滨工业大学的语义网络



待求解问题的语义网络片段

2.4.6 语义网络表示法的特点

□ 主要优点

- **结构性:** 采用把事物的属性以及事物间的各种语义联系显式地表示出来, 是一种结构化的知识表示方法。
- **联想性:** 本来是作为人类联想记忆模型提出来的, 它着重强调事物间的语义联系, 体现了人类的联想思维过程。
- **自索引性:** 把各接点之间的联系以明确、简洁的方式表示出来, 通过与某一结点连结的弧可以很容易的找出与该结点有关的信息, 而不必查找整个知识库。这种自索引能力有效的避免搜索时所遇到的组合爆炸问题。

➤ 主要缺点

- **非严格性:** 没有象谓词那样严格的形式表示体系, 一个给定语义网络的含义完全依赖于处理程序对它所进行的解释, 通过语义网络所实现的推理不能保证其正确性。
- **复杂性:** 语义网络表示知识的手段是多种多样的, 这虽然对其表示带来了灵活性, 但同时也由于表示形式的不一致, 使得它的处理增加了复杂性。

目 录

- 01 知识与知识表示
- 02 一阶谓词逻辑表示法
- 03 产生式表示法
- 04 语义网络表示法
- 05 框架表示法

2.5.1 框架表示法概述

- 1975年，Minsky提出了框架理论。他根据人们在理解情景、故事时提出的心理学模型，认为人的知识以框架结构存在人脑中。
- 认为人们对现实世界中各种事物的认识都是以一种类似于框架的结构存储在记忆中的，当遇到一个新事物时，就从记忆中找到一个合适的框架，并根据新的情况对其细节加以修改、补充，从而形成对这个新事物的认识。例如，对饭店、教室等的认识。

2.5.1 框架表示法概述

- **框架:** 是人们认识事物的一种通用的数据结构形式。即当新情况发生时, 人们只要把新的数据加入到该通用数据结构中, 便可形成一个具体的实体(类), 这样的通用数据结构就称为框架。
- **实例框架:** 对于一个框架, 当人们才把观察或认识到的具体细节填入后, 就得到了该框架的一个具体实例, 框架的这种具体实例被称为实例框架。
- **框架系统:** 在框架理论中, 框架是知识的基本单位, 把一组有关的框架连结起来可形成一个框架系统。
- **框架系统推理:** 由框架之间的协调来完成。

2.5.2 框架的组成

组成

- 一个“框架”由若干个“槽”组成，每个“槽”又划分为若干个“侧面”。

一个“槽”描述对象的一个方面属性；

一个“侧面”描述相应属性的一个方面。

由框架名、槽名、侧面、值组成。

2.5.2 框架的组成

一个框架结构为:

Frame<框架名>

槽名A:	侧面名A1	值A ₁₁ ,	值A ₁₂ ,	值A ₁₃ ...
	侧面名A2	值A ₂₁ ,	值A ₂₂ ,	值A ₂₃ ...
槽名B:	侧面名B1	值B ₁₁ ,	值B ₁₂ ,	值B ₁₃ ...
	侧面名B2	值B ₂₁ ,	值B ₂₂ ,	值B ₂₃ ...
槽名C:	侧面名C1	值C ₁₁ ,	值C ₁₂ ,	值C ₁₃ ...
	侧面名C2	值C ₂₁ ,	值C ₂₂ ,	值C ₂₃ ...
⋮	⋮	⋮	⋮	⋮

例：一个直接描述硕士生有关情况的框架

Frame <MASTER>

Name: Unit (Last-name, First-name)

Sex: Area (male, female)

Default: male

Age: Unit (Years)

Major: Unit (Major)

Field: Unit (Field)

Advisor: Unit (Last-name, First-name)

Project: Area (National, Provincial, Other)

Default: National

Paper: Area (SCI, EI, Core, General)

Default: Core

Address: < S-Address>

Telephone: Home Unit (Number)

Mobile Unit (Number)

对那些结构比较复杂的知识，往往需要用多个相互联系的框架来表示。例如，对前面硕士生框架“MASTER”可分为：

"Student"框架，描述所有学生的共性，上层框架

"Master"框架，描述硕士生的个性，子框架，继承"Student"框架的属性

学生框架

Frame <Student>

Name: Unit (Last-name, First-name)

Sex: Area (male, female)

Default: male

//缺省

Age: Unit (Years)

If-Needed: Ask-Age

//询问赋值

Address: < S-Address>

Telephone: Home Unit (Number)

Mobile Unit (Number)

If-Needed: Ask-Telephone

//询问赋值

硕士生框架

Frame <Master>

AKO: <Student>

//预定义槽名

Major: Unit (Major)

//专业

If-Needed: Ask - Major

//询问赋值

If-Added: Check-Major

//后继处理

Field: Unit (Field)

//方向

If-Needed : Ask - Field

//询问赋值

Advisor: Unit (Last-name, First-name)

//导师

If-Needed : Ask -Visor

//询问赋值

Project: Area (National, Provincial, Other)

//项目

Default: National

//缺省

Paper: Area (SCI, EI, Core, General)

//论文

Default: Core

//缺省

这里，用到了一个系统**预定义槽名AKO**，其含义为"是一种"。

当**AKO**作为下层框架的槽名时，其槽值为上层框架的框架名，表示该下层框架所描述的事物比其上层框架更具体。并且，由**AKO**所联系的框架之间具有属性的继承关系。

实例框架

例如，有杨叶和柳青2个硕士生，
杨叶，女，计算机专业，参加了导师林海的网络智能研究方向的省部级项目；
柳青，22岁，计算机专业，导师是林海，论文被EI收录。

硕士生-1框架:

Frame <Master-1>
ISA: <Master> //是一个
Name: Yang, Ye
Sex: female
Major: Computer
Field: Web-Intelligence //方向Web智能
Advisor: Lin Hai //导师林海
Project: Provincial //项目省部级

硕士生-2框架:

Frame <Master-2>
ISA: <Master>
Name: Liu, Qing
Age: 22
Major: Computer
Advisor: Lin Hai
Paper: EI //论文EI收录

其中用到了系统预定以槽名ISA，即Master-1和Master-2是2个具体的Master

2.5.3 框架系统

□基本结构

框架系统由框架之间的横向或纵向联系构成。

纵向联系

是指那种具有继承关系的上下层框架之间的联系。如:学生可按照接受教育的层次分为本、硕和博。每类学生又可按照所学专业的不同划分。

纵向联系通过预定义槽名**AKO**和**ISA**等来实现。

横向联系

是指那种以另外一个框架名作为一个槽的槽值或侧面值所建立起来的框架之间的联系。如**Student**框架与**S-Address**框架之间就是一种横向联系。

2.5.3 框架系统

特性继承

特性继承过程

通过ISA、AKO链来实现。

当需要查询某一事物的某个属性，且描述该事物的框架未提供其属性值时，系统就沿ISA和AKO链追溯到具有相同槽的类或超类框架。

如果该槽提供有Default侧面值，就继承该默认值作为查询结果返回。

如果该槽提供有If-Needed侧面供继承，则执行If-Needed操作，去产生一个值作为查询结果。

如果对某个事物的某一属性进行了赋值或修改操作，则系统会自动沿ISA和AKO链追溯到具有相应的类或超类框架，去执行If-Added操作，作相应的后继处理。

If-Needed与If-Added过程的区别

它们的主要区别在于激活时机和操作目的不同。

If-Needed操作是在系统试图查询某个事物框架中未记载的属性值时激活，并根据查询需求，被动地即时产生所需要的属性值；

If-Added操作是在系统对某个框架的属性作赋值或修改工作后激活，目的在于通过这些后继处理，主动做好配套操作，以消除可能存在的不一致。

2.5.3 框架系统

□ 特性继承的例

如前面的学生框架

若要查询Master-1 的Sex，则可直接回答；但要查询Master-2的Sex，则需要沿ISA链和AKO链到Student框架取其默认值male。

若要查询Master-2的Field，需要沿ISA链到Master框架，执行Field槽If-Needed

侧面的Ask-Field操作，即时产生一个值，假设产生的值是Data-Mining，则表示Master-2的研究方向为数据挖掘。

如果要修改Master-2 的Major，需要沿ISA链到Master框架，执行Major槽If-Added侧面的Check-Major操作，对Field, Advisor进行修改，以保持知识的一致性。

2.5.4 框架系统的问题求解过程

□ 匹配和填槽

框架的匹配实际上是通过对应槽的槽名和槽值逐个进行比较，并利用继承关系来实现的。

例如，假设前面讨论的学生框架系统已建立在知识库中，若要求从知识库中找出一个满足如下条件的硕士生：

male, Age<25 , Major为Computer , Project为National

把这些条件用框架表示出来，就可得到如下的初始问题框架

Frame <Master-x>

Name:

Sex: male

Age: Years <25

Major: Computer

Project: National

用此框架和知识库中的框架匹配，显然“Master -2”框架可以匹配。因为Age、Major槽都符合要求，Sex 槽和Project槽虽然没有给出，但由继承性可知它们分别取默认值male和National，完全符合初始问题框架Master-x的要求，所以要找的学生有可能是Liu Qing。

2.5.4 框架系统的问题求解过程

例： 请用框架表示这一知识： 范伟， 男， 30岁, 1996年10月到2012年8月间在计算机学院任讲师。

2.5.4 框架系统的问题求解过程

例： 请用框架表示这一知识： 范伟， 男， 30岁, 1996年10月到2012年8月间在计算机学院任讲师。

```
Frame: <Teacher-1>
Name:  Fan, Wei
Sex:   Male
Age:   30
Job:   Lecturer
Work-time:
    Start: 1996-10
    End:   2012-08
Department:  Computer Science
```

2.5.5 框架表示法的特征

□ 主要优点:

- **结构性**:最突出特点是善于表示结构性知识, 它能够把知识的内部结构关系以及知识间的特殊联系表示出来。
- **深层性**:框架表示法不仅可以从多个方面、多重属性表示知识, 而且还可以通过ISA、AKO等槽以嵌套结构分层地对知识进行表示, 因此能用来表达事物间复杂的深层联系。
- **继承性**:在框架系统中, 下层框架可以继承上层框架的槽值, 也可以进行补充和修改, 这样既减少知识冗余, 又较好地保证了知识的一致性。
- **自然性**:框架能把与某个实体或实体集相关特性都集中在一起, 从而高度模拟了人脑对实体多方面、多层次的存储结构, 直观, 自然, 易于理解。

□ 主要缺点

- **缺乏框架的形式理论**:至今, 还没有建立框架的形式理论。
- **缺乏过程性知识表示**:框架系统不适于表示过程性知识, 缺乏如何使用框架中知识的描述能力。框架推理过程需要用到一些与领域无关的推理规则, 而这些规则在框架系统中又很难表达。
- **清晰性难以保证**:由于各框架本身的数据结构不一定相同, 从而框架系统的清晰性很难保证。

第二章 课下作业

2.10, 2.16, 2.23

谢 谢!